UNIVERSIDADE FEDERAL DE ALAGOAS

INSTITUTO DE COMPUTAÇÃO

CIÊNCIA DA COMPUTAÇÃO

ÉRICO ALMEIDA BERNARDINO

FELIPE LIMA ARAGÃO

LUCAS TRINDADE DE SOUZA

DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR 2026.2

https://github.com/Felipe-Aragao/Minipar2026.2

MACEIÓ – AL

2026

SUMÁRIO

1 INTRODUÇÃO 4
1.1 Objetivos do Projeto 5
1.2 Arquitetura Geral do Pipeline de Compilação 5

2 A LINGUAGEM MINIPAR 2026.2 E GRAMÁTICA BNF 7
2.1 Recursos de Paralelismo 7
2.2 Gramática Formal em Notação EBNF 8
2.2.1 Estrutura Global do Programa 8
2.2.2 Comandos e Estruturas de Controle 8
2.2.3 Expressões e Precedência de Operadores 9

3 ANALISADOR LÉXICO (LEXER) 11
3.1 Tratamento de Comentários, Delimitadores e Literais 11
3.2 Pseudocódigo do Analisador Léxico 11

4 ANALISADOR SINTÁTICO (PARSER) 15
4.1 Método de Descida Recursiva e Precedência por Camadas 15
4.2 Tratamento dos Construtos `fork` e `join` 15
4.3 Pseudocódigo do Analisador Sintático 16

5 ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS 19
5.1 Estratégia de Duas Passagens (*Two-Pass Analysis*) 19
5.2 Tratamento de Comentários (`#`) e Diagnósticos de Erros 19
5.3 Pseudocódigo do Analisador Semântico 20

6 TABELA DE SÍMBOLOS 23
6.1 Estrutura de Dados e Pilha de Escopos Léxicos 23
6.2 Pseudocódigo da Tabela de Símbolos 24

7 MOTOR DE OTIMIZAÇÃO NO BACKEND 26
7.1 Técnicas de Otimização Implementadas 26
7.2 Ponto Fixo **27**
7.3 Métricas e Ativação no Compilador 27

8 GERAÇÃO DE CÓDIGO INTERMEDIÁRIO (TAC) 28
8.1 Estrutura do TAC e Linearização de Controle 28

8.2 Verificação do TAC no Teste 7 (Quicksort Iterativo) **28**

9 GERAÇÃO DE CÓDIGO C++20 E RUNTIME NATIVA 31
9.1 Transpilação da AST para C++20 31
9.2 Runtime Nativa e Afinidade de Núcleos (*Core Affinity*) 32
9.2.1 Isolamento de Núcleos de CPU (Afinidade de Hardware) 32
9.2.2 Pool de Tarefas e Barreiras Assíncronas 32
9.2.3 Comunicação TCP com Resiliência 32
9.3 Compilação Nativa com g++ 33

10 INTERFACES DO COMPILADOR (3 MODALIDADES) 34
10.1 Modalidade 1: Interface de Linha de Comando (CLI) e Menu Terminal 34
10.2 Modalidade 2: Primeira Interface Web (Desenvolvedor) 34
10.3 Modalidade 3: Segunda Interface Web (Didática / Educacional) 35
10.3.1 Inspiração Visual e Filosofia de Design 35
10.3.2 Controles Didáticos Passo a Passo (Botões de Fase) 35

11 GERÊNCIA DE PROJETO ÁGIL (PRODUCT E SPRINTS BACKLOG) 40
11.1 Product Backlog 40
11.2 Sprints Backlog 40

12 MODELAGEM UML BASEADA EM COMPONENTES DE SOFTWARE (CBSE) 43
12.1 Diagrama de Casos de Uso 43
12.2 Arquitetura de Componentes de Software 43
12.3 Diagramas de Classes por Componente 44

13 TESTES DE INTEGRAÇÃO DE COMPONENTES DE SOFTWARE . 49
13.1 Matriz de Integração dos Componentes 49
13.2 Resultados da Suíte Automatizada de Testes (Pytest) 49

14 BATERIA OFICIAL DOS 11 PROGRAMAS DE TESTE 51
14.1 Teste 1: Calculadora Cliente-Servidor com Sockets TCP 51
14.2 Teste 2: Paralelismo Concorrente (Fatorial e Fibonacci) 51
14.3 Teste 3: Neurônio Perceptron com Orientação a Objetos 52
14.4 Teste 4: Rede Neural MLP (Problema XOR) 52
14.5 Teste 5: Sistema de Recomendação de E-commerce 53
14.6 Teste 6: Quicksort Recursivo com POO 54
14.7 Teste 7: Quicksort Iterativo com Pilha Explícita e Validação de TAC . . 55
14.8 Teste 8: Avaliação Paralela em 4 Núcleos de CPU (Speedup) 56

14.9 **Teste 9: 5 Núcleos (Avaliação Paralela + Fractal Mandelbrot ASCII)** . . 57
14.10 **Teste 10: 6 Núcleos (Avaliação + Mandelbrot + Tapete de Sierpinski)** . . 58
14.11 **Teste 11: Tokenizador, Avaliador e Mini-Compilador Assembly ARMv7** 59
14.12 **Tabela Resumo dos Estados dos Testes** 61

15 **IMPLEMENTAÇÃO E AMBIENTE DE DESENVOLVIMENTO . . . .** 62
15.1 **Tecnologias Utilizadas e Ambiente** 62
15.1.1 Comentários Explicativos no Código-Fonte 62
15.2 **Passo a Passo da Execução dos Programas de Teste** 62

16 **CONCLUSÃO . . . . . .** 64

LINKS ÚTEIS 65

REFERÊNCIAS 66

Apêndice D - Registros de execução em rede 67

1 INTRODUÇÃO

Este relatório técnico documenta a concepção, modelagem arquitetural, implementação e validação experimental do compilador para a linguagem MiniPar 2026.2, desenvolvido no âmbito da disciplina de Compiladores ministrada pelo Prof. Dr. Arturo Hernandez Dominguez no Instituto de Computação da Universidade Federal de Alagoas (UFAL).

A linguagem MiniPar é uma linguagem educacional projetada para explorar conceitos avançados de compilação, orientação a objetos e execução concorrente e distribuída. O MiniPar 2026.2 adota um pipeline estático composto por análise léxica, análise sintática com geração de Árvore de Sintaxe Abstrata (AST), análise semântica com verificação de tipos e escopos, motor de otimização independente de máquina na AST, síntese de Código Intermediário de Três Endereços (TAC) e transpilação para C++ nativo (padrão ISO C++20), gerando binários executáveis (`.exe`) compilados pelo `g++` com a flag `-O3`.

Partimos do MiniPar 2026.1 e acrescentamos recursos para atender às

exigências da disciplina:

- Arquitetura Baseada em Componentes (CBSE): desacoplamento de todas as etapas em componentes formais com interfaces abstratas no diretório `src/` (`componente_lexico.py`, `componente_sintatico.py`, `componente_semantico.py`, `componente_tabela_simbolos.py`, `componente_otimizador.py`, `componente_tac.py` e `componente_codegen.py`);
- Primitivas de Real Paralelismo (`fork` e `join`): expansão dos analisadores léxico, sintático, semântico e do gerador de TAC para divisão explícita de tarefas em núcleos locais (`core`) ou nós remotos (`host`), com barreira de sincronização;
- Afinidade de Núcleos de Hardware e Runtime C++: chamadas nativas de sistema operacional em `src/cpp_template.py` (`SetThreadAffinityMask` na API Win32 e `pthread_setaffinity_np` em POSIX/Linux) e pool assíncrono com `std::shared_future`;
- Motor de Otimização no Backend: módulo `src/optimizer.py` para simplificações algébricas, dobra de constantes (*constant folding*) e eliminação de código morto na AST;
- Segunda Interface Web Didática: interface em `templates/didatico.html` com visualizador gráfico interativo da AST em SVG/D3.js, controles individuais para cada fase da compilação e suporte a entradas pré-digitadas no console;
- Suíte e Orquestrador dos 11 Testes Oficiais: validação dos 11 programas oficiais da disciplina, cobrindo cálculo de Speedup com multiprocessamento, renderização de fractais (Mandelbrot e Tapete de Sierpinski em BMP) e um mini-compilador com emissão de Assembly ARMv7 no Teste 11.

1.1 Objetivos do Projeto

O trabalho cumpre os requisitos da especificação oficial do projeto MiniPar 2026.2, assegurando:

1. Corretude e Confiabilidade: compilação e execução estável dos 11 testes oficiais e aprovação total na suíte automatizada de testes;
2. Rigor de Engenharia de Software: modelagem formal da arquitetura em diagramas UML de Casos de Uso, Componentes de Software (interfaces fornecidas e requeridas), Classes Conceituais e Atividades/Fluxo;
3. Didática e Usabilidade: disponibilização de interfaces acessíveis tanto para desenvolvedores quanto para estudantes em aprendizado de compiladores.

1.2 Arquitetura Geral do Pipeline de Compilação

A Tabela 1 resume a estrutura arquitetural dos módulos do sistema, detalhando suas responsabilidades, arquivos de implementação e interfaces associadas.

Tabela 1 – Mapeamento da Arquitetura Geral do Compilador MiniPar 2026.2

Componente	Arquivo de Implementação	Interface (CBSE)	Responsabilidade Principal
Entrada CLI	`main.py`	CLI / Dispatcher	Ponto de entrada e controle de flags (`-O`) e seletor de modo interativo.
Menu Terminal	`menu.py`	Console UI	Execução individual ou em lote dos 11 testes oficiais.
Servidor Web Flask	`app.py`	Web REST API	Disponibiliza a IDE Desenvolvedor (`/ide`) e a Didática (`/`).
Analisador Léxico	`src/lexer.py`	`ILexico`	Converte texto-fonte em tokens com linha e coluna, descartando comentários `#`.
Analisador Sintático	`src/parser.py`	`ISintatico`	Constrói a AST com nós orientados a objetos via descida recursiva.
Nós da AST	`src/ast_nodes.py`	Dataclasses	Modela nós sintáticos e oferece serialização JSON via `to_dict()`.
Tabela de Símbolos	`src/componente_tabela_simbolos.py`	`ITabelaSimbolos`	Controla a pilha de escopos, variáveis, atributos e métodos.
Analisador Semântico	`src/semantic.py`	`ISemantico`	Validação estática de tipos, escopos, classes, herança e laços.
Otimizador AST	`src/optimizer.py`	`IOtimizador`	Backend: dobra de constantes, simplificação algébrica e código morto na AST.
Gerador de TAC	`src/tac.py`	`ITAC`	Síntese de código linear de 3 endereços com temporários e labels.
Gerador de C++	`src/codegen.py`	`ICodegen`	Transpila AST validada para C++20 nativo e invoca o `g++ -O3`.
Runtime C++	`src/cpp_template.py`	Header nativo C++	Threads, sockets TCP, afinidade de CPU e barreira de sincronização.
Análise Web	`src/analise.py`	Engine Modular	Executa o pipeline passo a passo em memória para a interface didática.
Orquestrador Multi-Terminal	`test_orchestrator.py`	Sockets / JSON	Coordena terminais participantes em testes distribuídos.

2 A LINGUAGEM MINIPAR 2026.2 E GRAMÁTICA BNF

A linguagem MiniPar 2026.2 é uma linguagem imperativa, concorrente, distribuída e fortemente tipada em seu modelo conceitual, integrando elementos sintáticos inspirados em Python, C++ e Go. Sua gramática formal suporta abstrações para Orientação a Objetos, controle de fluxo, manipulação de coleções dinâmicas e estruturas para comunicação e paralelismo.

2.1 Recursos de Paralelismo

O compilador executa tarefas em múltiplos núcleos e nós de rede:

1. Blocos `seq` e `par`: O comando `seq { ... }` garante a ordem sequencial de execução dos enunciados internos. O bloco `par { ... }` dispara os comandos contidos como threads concorrentes no computador local, aguardando que todas concluam antes de prosseguir.

2. Primitivas `fork` e `join` (Paralelismo Real Distribuído): A primitiva `fork` realiza a divisão de uma tarefa em subtarefas independentes alocadas explicitamente a recursos de processamento:

    - `fork core(id) comando;` ou `fork core(id) { ... }`: realiza a alocação determinística da subtarefa a um núcleo de CPU do processador local, isolando sua execução via afinidade de hardware (*Core Affinity*).

• fork host(ip, porta) envia uma chamada de função a um worker via TCP.

A primitiva complementar `join` implementa a barreira de sincronização e coleta de dados:

- `join;` : aguarda o término de todas as tarefas disparadas pelo pool local.
- `join(t1, t2, ...);` : bloqueia a execução até que as tarefas indicadas concluam e disponibilizem seus resultados.

3. Comunicação por Mensagens via Canais (`c_channel`): A troca de dados entre processos independentes ocorre por passagem de mensagens utilizando canais sobre sockets TCP/IP:

    - Declaração: `var canal = c_channel("127.0.0.1", 5000);`
    - Envio: `send(canal, mensagem);`
    - Recepção: `var dado = receive(canal);`

4. Orientação a objetos: classes, herança simples, instanciação com new e referência this.
   Métodos aceitam parâmetros. O gerador não emite construtores parametrizados próprios.

A sintaxe formal completa do MiniPar 2026.2 é descrita na notação BNF Estendida (EBNF), na qual `{ x }` denota zero ou mais repetições, `[ x ]` denota elemento opcional e `"token"` representa símbolo terminal.

2.2.1 Estrutura Global do Programa

```
<programa> ::= { <declaracao> | <comando> }

<declaracao> ::= <declaracao_classe>
               | <declaracao_funcao>
               | <declaracao_variavel>

<declaracao_classe> ::= "class" <IDENTIFIER> [ "extends" <IDENTIFIER> ] "{"
                          { <membro_classe> }
                        "}"

<membro_classe> ::= <declaracao_variavel>
                  | <declaracao_funcao>

<declaracao_funcao> ::= "func" <IDENTIFIER> "(" [ <lista_parametros> ] ")" <
    bloco_comandos>

<lista_parametros> ::= <IDENTIFIER> { "," <IDENTIFIER> }

<declaracao_variavel> ::= ( "var" | <tipo> ) <IDENTIFIER> [ "=" <expressao> ] ";"
```

Listing 2.1 – Produções globais e declarações

2.2.2 Comandos e Estruturas de Controle

```
<comando> ::= <bloco_comandos>
            | <bloco_seq>
            | <bloco_par>
            | <comando_fork>
            | <comando_join>
            | <comando_if>
            | <comando_while>
            | <comando_do_while>
            | <comando_for>
```

```
            | <comando_print>
            | <comando_send>
            | <comando_return>
            | <comando_break>
            | <comando_continue>
            | <atribuicao> ";"
            | <chamada_metodo> ";"
            | <chamada_funcao> ";"

<bloco_comandos> ::= "{" { <comando> } "}"
<bloco_seq>      ::= "seq" "{" { <comando> } "}"
<bloco_par>      ::= "par" "{" { <comando> } "}"

<comando_fork>   ::= "fork" <destino> ( <bloco_comandos> | <expressao> ";" )
<destino>        ::= "core" "(" <expressao> ")"
                   | "host" "(" <expressao> [ "," <expressao> ] ")"

<comando_join>   ::= "join" [ "(" [ <lista_expressoes> ] ")" ] ";"

<comando_if>     ::= "if" "(" <expressao> ")" <comando> [ "else" <comando> ]

<comando_while>  ::= "while" "(" <expressao> ")" <comando>

<comando_do_while> ::= "do" <comando> "while" "(" <expressao> ")" ";"

<comando_for>    ::= "for" "(" ( <for_tradicional> | <for_in> ) ")" <comando>
<for_tradicional> ::= [ <declaracao_variavel> | <atribuicao> ] ";"
                      [ <expressao> ] ";"
                      [ <atribuicao> | <expressao> ]
<for_in>          ::= "var" <IDENTIFIER> "in" <expressao>

<comando_print>  ::= "print" "(" [ <lista_expressoes> ] ")" ";"
<comando_send>   ::= "send" "(" <expressao> "," <expressao> ")" ";"
<comando_return> ::= "return" [ <expressao> ] ";"
<comando_break>  ::= "break" ";"
<comando_continue> ::= "continue" ";"
```

Listing 2.2 – Produções de comandos e controle de fluxo

2.2.3 Expressões e Precedência de Operadores

```
<expressao> ::= <expressao_logica_ou>

<expressao_logica_ou> ::= <expressao_logica_e> { "||" <expressao_logica_e> }

<expressao_logica_e>  ::= <expressao_igualdade> { "&&" <expressao_igualdade> }
```

```
<expressao_igualdade> ::= <expressao_relacional> { ( "==" | "!=" ) <
    expressao_relacional> }

<expressao_relacional> ::= <expressao_aditiva> { ( "<" | "<=" | ">" | ">=" ) <
    expressao_aditiva> }

<expressao_aditiva>   ::= <expressao_multiplicativa> { ( "+" | "-" ) <
    expressao_multiplicativa> }

<expressao_multiplicativa> ::= <expressao_unaria> { ( "*" | "/" | "%" ) <
    expressao_unaria> }

<expressao_unaria>    ::= ( "-" | "!" ) <expressao_unaria> | <expressao_posfixa>

<expressao_posfixa>   ::= <expressao_primaria> {
                            "." <IDENTIFIER> [ "(" [ <lista_expressoes> ] ")" ]
                          | "[" <expressao> "]"
                        }

<expressao_primaria>  ::= <NUMBER>
                        | <STRING_LITERAL>
                        | <BOOL_LITERAL>
                        | <IDENTIFIER>
                        | "this"
                        | "new" <IDENTIFIER> "(" [ <lista_expressoes> ] ")"
                        | "c_channel" "(" <expressao> "," <expressao> ")"
                        | "receive" "(" <expressao> ")"
                        | "input" "(" [ <expressao> ] ")"
                        | "matrix" "(" <expressao> "," <expressao> "," <expressao> "
    )"
                        | "[" [ <lista_expressoes> ] "]"
                        | "(" <expressao> ")"
                        | <comando_fork>
                        | <comando_join>
```

Listing 2.3 – Hierarquia sintática de expressões

Nota sobre a gramática. A BNF completa do repositório é a referência normativa. O destino de fork é opcional. Em expressões, fork e join não incluem o ponto e vírgula do comando. Blocos aceitam declarações e o cabeçalho de for mantém dois separadores.

3 ANALISADOR LÉXICO (LEXER)

O Analisador Léxico (*Lexer*) é o primeiro estágio do front-end do compilador MiniPar 2026.2, implementado no arquivo `src/lexer.py` e encapsulado no componente `C_Lexico` que realiza a interface `ILexico`.

Sua responsabilidade é varrer sequencialmente o fluxo de caracteres do código-fonte e convertê-lo em uma lista linear de objetos estruturados da classe `Token`, na qual cada token carrega seu tipo categórico (`TokenType`), o lexema correspondente em texto, o número da linha de ocorrência e a coluna exata.

3.1 Tratamento de Comentários, Delimitadores e Literais

O analisador léxico distingue comentários, delimitadores e literais.
As regras são as seguintes:

- Filtragem de Comentários (`#`): ao encontrar o caractere cerquilha (`#`) fora de literais de texto, o analisador consome todos os caracteres até a quebra de linha (`\n`) ou o fim do arquivo. Isso assegura que anotações e comentários não interfiram nas fases sintáticas posteriores.
- Rastreamento Espacial: o contador de linha inicia em 1 e é incrementado a cada `\n`. A coluna é atualizada a cada caractere lido e reiniciada após quebras de linha.
- Literais Numéricos e Ponto Flutuante: reconhece inteiros (`42`) e números decimais (`3.14159`), consumindo sequências de dígitos separadas por ponto.
- Literais de Texto e Escapes: delimitados por aspas duplas (`"..."`), com suporte a sequências de escape padrão (`\n`, `\t`, `\"` e `\\`). O não fechamento das aspas dispara `LexError` com linha e coluna.
- Operadores Compostos: utiliza *lookahead* de um caractere para diferenciar operadores simples de relacionais e lógicos compostos (==, `!=`, <=, >=, `&&` e `||`).

A Tabela 2 resume as categorias de tokens reconhecidas pelo compilador.

3.2 Pseudocódigo do Analisador Léxico

O Algoritmo 3.1 formaliza o procedimento de tokenização adotado pelo MiniPar 2026.2, atendendo expressamente ao Requisito 3 da especificação.

```
ALGORITMO ANALISE_LEXICA(codigo_fonte):
    posicao <- 0
    linha <- 1
```

Tabela 2 – Categorias e Exemplos de Tokens no MiniPar 2026.2

Categoria	Tokens Principais
Palavras Reservadas	`class`, `extends`, `new`, `func`, `var`, `if`, `else`, `while`, `do`, `for`, `in`, `break`, `continue`, `return`, `seq`, `par`, `fork`, `join`, `core`, `host`, `this`, `c_channel`, `send`, `receive`, `input`, `print`, `matrix`
Tipos Nativos	`number`, `int`, `string`, `bool`, `void`, `list`, `dict`, `task`
Literais	`NUMBER`, `STRING_LITERAL`, `BOOL_LITERAL` (`true`, `false`), `IDENTIFIER`
Operadores	`+`, `-`, `*`, `/`, `%`, `=`, `==`, `!=`, `!`, `&&`, `||`, `<`, `>`, `<=`, `>=`
Delimitadores	`(`, `)`, `{`, `}`, `[`, `]`, `.`, `;`, `,`
Especiais	`END_OF_FILE`, `UNKNOWN` (para caracteres não reconhecidos)

```
coluna <- 1
tokens <- Lista_Vazia()

ENQUANTO posicao < tamanho(codigo_fonte):
    # 1. Pular espacos em branco e tabulacoes
    ENQUANTO posicao < tamanho(codigo_fonte) E eh_espaco(codigo_fonte[posicao]):
        SE codigo_fonte[posicao] == '\n':
            linha <- linha + 1
            coluna <- 1
        SENAO:
            coluna <- coluna + 1
        posicao <- posicao + 1

    # 2. Descarte de comentarios iniciados por '#'
    SE posicao < tamanho(codigo_fonte) E codigo_fonte[posicao] == '#':
        ENQUANTO posicao < tamanho(codigo_fonte) E codigo_fonte[posicao] != '\n'
:
            posicao <- posicao + 1
            coluna <- coluna + 1
        CONTINUAR_PROXIMA_ITERACAO

    SE posicao >= tamanho(codigo_fonte):
        PARAR_LACO

    inicio_linha <- linha
    inicio_coluna <- coluna
    caractere <- codigo_fonte[posicao]

    # 3. Identificadores e Palavras Reservadas
    SE eh_letra(caractere) OU caractere == '_':
        lexema <- ""
        ENQUANTO posicao < tamanho(codigo_fonte) E (eh_alfanumerico(codigo_fonte
[posicao]) OU codigo_fonte[posicao] == '_'):
            lexema <- lexema + codigo_fonte[posicao]
```

```
            posicao <- posicao + 1
            coluna <- coluna + 1

        tipo <- CONSULTAR_PALAVRAS_RESERVADAS(lexema)
        SE tipo == NULO:
            tipo <- TOKEN_IDENTIFICADOR
        ADICIONAR_TOKEN(tokens, tipo, lexema, inicio_linha, inicio_coluna)

    # 4. Numeros (Inteiros e Flutuantes)
    SENAO SE eh_digito(caractere):
        lexema <- ""
        ENQUANTO posicao < tamanho(codigo_fonte) E eh_digito(codigo_fonte[
posicao]):
            lexema <- lexema + codigo_fonte[posicao]
            posicao <- posicao + 1
            coluna <- coluna + 1

        SE posicao < tamanho(codigo_fonte) E codigo_fonte[posicao] == '.':
            lexema <- lexema + '.'
            posicao <- posicao + 1
            coluna <- coluna + 1
            ENQUANTO posicao < tamanho(codigo_fonte) E eh_digito(codigo_fonte[
posicao]):
                lexema <- lexema + codigo_fonte[posicao]
                posicao <- posicao + 1
                coluna <- coluna + 1

        ADICIONAR_TOKEN(tokens, TOKEN_NUMERO, lexema, inicio_linha,
inicio_coluna)

    # 5. Strings Literais e Caracteres de Escape
    SENAO SE caractere == '"':
        posicao <- posicao + 1  # Consome aspas de abertura
        coluna <- coluna + 1
        texto <- ""
        ENQUANTO posicao < tamanho(codigo_fonte) E codigo_fonte[posicao] != '"':
            SE codigo_fonte[posicao] == '\\':
                posicao <- posicao + 1
                coluna <- coluna + 1
                SE posicao >= tamanho(codigo_fonte):
                    LANCAR_ERRO_LEXICO("String nao terminada", inicio_linha,
inicio_coluna)
                escape_char <- codigo_fonte[posicao]
                texto <- texto + PROCESSAR_ESCAPE(escape_char)
            SENAO:
                texto <- texto + codigo_fonte[posicao]
            posicao <- posicao + 1
```

```
                coluna <- coluna + 1

            SE posicao >= tamanho(codigo_fonte) OU codigo_fonte[posicao] != '"':
                LANCAR_ERRO_LEXICO("String nao terminada", inicio_linha,
    inicio_coluna)

            posicao <- posicao + 1  # Consome aspas de fechamento
            coluna <- coluna + 1
            ADICIONAR_TOKEN(tokens, TOKEN_STRING, texto, inicio_linha, inicio_coluna
    )

        # 6. Operadores Relacionais, Compostos e Delimitadores
        SENAO:
            simbolo <- AVANCAR_E_OBTER_CARACTERE()
            SE simbolo == '=' E ESPIAR_PROXIMO() == '=':
                AVANCAR(); ADICIONAR_TOKEN(tokens, TOKEN_IGUALDADE, "==",
    inicio_linha, inicio_coluna)
            SENAO SE simbolo == '!' E ESPIAR_PROXIMO() == '=':
                AVANCAR(); ADICIONAR_TOKEN(tokens, TOKEN_DIFERENTE, "!=",
    inicio_linha, inicio_coluna)
            SENAO SE simbolo == '<' E ESPIAR_PROXIMO() == '=':
                AVANCAR(); ADICIONAR_TOKEN(tokens, TOKEN_MENOR_IGUAL, "<=",
    inicio_linha, inicio_coluna)
            SENAO SE simbolo == '>' E ESPIAR_PROXIMO() == '=':
                AVANCAR(); ADICIONAR_TOKEN(tokens, TOKEN_MAIOR_IGUAL, ">=",
    inicio_linha, inicio_coluna)
            SENAO SE simbolo == '&' E ESPIAR_PROXIMO() == '&':
                AVANCAR(); ADICIONAR_TOKEN(tokens, TOKEN_E_LOGICO, "&&",
    inicio_linha, inicio_coluna)
            SENAO SE simbolo == '|' E ESPIAR_PROXIMO() == '|':
                AVANCAR(); ADICIONAR_TOKEN(tokens, TOKEN_OU_LOGICO, "||",
    inicio_linha, inicio_coluna)
            SENAO SE simbolo ESTA_NO_MAPA_SIMBOLOS_SIMPLES():
                ADICIONAR_TOKEN(tokens, MAPA_SIMBOLOS[simbolo], simbolo,
    inicio_linha, inicio_coluna)
            SENAO:
                ADICIONAR_TOKEN(tokens, TOKEN_DESCONHECIDO, simbolo, inicio_linha,
    inicio_coluna)

    ADICIONAR_TOKEN(tokens, TOKEN_FIM_ARQUIVO, "", linha, coluna)
    RETORNAR tokens
FIM_ALGORITMO
```

Listing 3.1 – Pseudocódigo do Analisador Léxico (Lexer)

4 ANALISADOR SINTÁTICO (PARSER)

O Analisador Sintático (*Parser*) é responsável por verificar se a sequência linear de tokens produzida pelo Lexer obedece estritamente às regras gramaticais da linguagem MiniPar 2026.2 e por estruturar essas informações em uma Árvore de Sintaxe Abstrata (AST).

No compilador MiniPar 2026.2, o analisador está implementado em `src/parser.py` e encapsulado no componente `C_Sintatico`, que realiza a interface `ISintatico`. A AST resultante é composta por classes de nós fortemente tipadas, definidas em `src/ast_nodes.py`.

4.1 Método de Descida Recursiva e Precedência por Camadas

O analisador utiliza a técnica de Análise Sintática Descendente Recursiva (*Recursive Descent Parsing*):

- Cada símbolo não-terminal da gramática BNF corresponde a uma função ou método dedicado na classe `Parser`.
- Para eliminar ambiguidades e respeitar a precedência matemática e lógica dos operadores sem recursão à esquerda, as expressões são tratadas em camadas estratificadas descendentes:

$$\text{Expressão} \longrightarrow \text{OU} \longrightarrow \text{E} \longrightarrow \text{Igualdade} \longrightarrow \text{Relacional}$$
$$\longrightarrow \text{Aditiva} \longrightarrow \text{Multiplicativa} \longrightarrow \text{Unária} \longrightarrow \text{Pós-fixa} \longrightarrow \text{Primária}$$

- Em caso de divergência entre o token esperado e o token lido, o método `_consume` lança uma exceção `ParseError`, indicando a mensagem acompanhada da linha e coluna correspondentes.

4.2 Tratamento dos Construtos `fork` e `join`

Foram adicionadas produções no analisador sintático para o mapeamento das primitivas de paralelismo:

1. Produção de `fork`: o método `_parse_fork()` identifica a palavra-chave `fork`, analisa o destino de alocação (`core(expressao)` ou `host(ip, porta)`) e constrói o nó `ForkStmt`, associando-o ao comando ou bloco correspondente.
2. Produção de `join`: o método `_parse_join()` reconhece a barreira global de sincronização (`join;`) e a espera seletiva por tarefas específicas (`join(t1, t2);`), gerando a instância de `JoinStmt`.

4.3 Pseudocódigo do Analisador Sintático

O Algoritmo 4.1 apresenta o pseudocódigo formal do analisador sintático, cumprindo o Requisito 4 do projeto.

```
CLASSE Parser:
    ATRIBUTOS:
        tokens: Lista de Tokens
        posicao: Inteiro <- 0

    METODO analisar_programa():
        programa <- Novo No Program()
        ENQUANTO token_atual().tipo != TOKEN_FIM_ARQUIVO:
            comando <- analisar_comando_ou_declaracao()
            SE comando != NULO:
                programa.adicionar_comando(comando)
        RETORNAR programa

    METODO consumir(tipo_esperado, mensagem_erro):
        SE token_atual().tipo == tipo_esperado:
            t <- token_atual()
            posicao <- posicao + 1
            RETORNAR t
        SENAO:
            LANCAR ParseError(mensagem_erro, token_atual().linha, token_atual().
    coluna)

    METODO analisar_comando_ou_declaracao():
        token <- token_atual()
        SE token.tipo == TOKEN_CLASS:
            RETORNAR analisar_declaracao_classe()
        SENAO SE token.tipo == TOKEN_FUNC:
            RETORNAR analisar_declaracao_funcao()
        SENAO SE token.tipo == TOKEN_VAR OU token.tipo == TOKEN_TIPO:
            RETORNAR analisar_declaracao_variavel()
        SENAO SE token.tipo == TOKEN_SEQ:
            RETORNAR analisar_bloco_seq()
        SENAO SE token.tipo == TOKEN_PAR:
            RETORNAR analisar_bloco_par()
        SENAO SE token.tipo == TOKEN_FORK:
            RETORNAR analisar_comando_fork()
        SENAO SE token.tipo == TOKEN_JOIN:
            RETORNAR analisar_comando_join()
        SENAO SE token.tipo == TOKEN_IF:
            RETORNAR analisar_comando_if()
        SENAO SE token.tipo == TOKEN_WHILE:
            RETORNAR analisar_comando_while()
```

```
    SENAO SE token.tipo == TOKEN_DO:
        RETORNAR analisar_comando_do_while()
    SENAO SE token.tipo == TOKEN_FOR:
        RETORNAR analisar_comando_for()
    SENAO SE token.tipo == TOKEN_PRINT:
        RETORNAR analisar_comando_print()
    SENAO SE token.tipo == TOKEN_SEND:
        RETORNAR analisar_comando_send()
    SENAO SE token.tipo == TOKEN_RETURN:
        RETORNAR analisar_comando_return()
    SENAO SE token.tipo == TOKEN_BREAK:
        consumir(TOKEN_BREAK, "Esperado 'break'"); consumir(TOKEN_PONTO_VIRGULA,
 "Esperado ';'")
        RETORNAR Novo No BreakStmt()
    SENAO SE token.tipo == TOKEN_CONTINUE:
        consumir(TOKEN_CONTINUE, "Esperado 'continue'"); consumir(
TOKEN_PONTO_VIRGULA, "Esperado ';'")
        RETORNAR Novo No ContinueStmt()
    SENAO:
        expr <- analisar_expressao()
        SE token_atual().tipo == TOKEN_ATRIBUICAO:
            consumir(TOKEN_ATRIBUICAO, "Esperado '='")
            valor <- analisar_expressao()
            consumir(TOKEN_PONTO_VIRGULA, "Esperado ';'")
            RETORNAR Novo No Assignment(alvo=expr, valor=valor)
        consumir(TOKEN_PONTO_VIRGULA, "Esperado ';'")
        RETORNAR expr

METODO analisar_comando_fork():
    consumir(TOKEN_FORK, "Esperado 'fork'")
    SE token_atual().tipo == TOKEN_CORE:
        consumir(TOKEN_CORE, "Esperado 'core'")
        consumir(TOKEN_ABRE_PARENTESES, "Esperado '('")
        nucleo <- analisar_expressao()
        consumir(TOKEN_FECHA_PARENTESES, "Esperado ')'")
        destino <- Novo No Destino(tipo="core", args=[nucleo])
    SENAO SE token_atual().tipo == TOKEN_HOST:
        consumir(TOKEN_HOST, "Esperado 'host'")
        consumir(TOKEN_ABRE_PARENTESES, "Esperado '('")
        host_ip <- analisar_expressao()
        consumir(TOKEN_VIRGULA, "Esperado ','")
        porta <- analisar_expressao()
        consumir(TOKEN_FECHA_PARENTESES, "Esperado ')'")
        destino <- Novo No Destino(tipo="host", args=[host_ip, porta])
    SENAO:
        LANCAR ParseError("Destino do fork deve ser 'core' ou 'host'",
token_atual().linha)
```

```
        corpo <- analisar_comando_ou_declaracao()
        RETORNAR Novo No ForkStmt(destino=destino, corpo=corpo)

    METODO analisar_comando_join():
        consumir(TOKEN_JOIN, "Esperado 'join'")
        args <- Lista_Vazia()
        SE token_atual().tipo == TOKEN_ABRE_PARENTESES:
            consumir(TOKEN_ABRE_PARENTESES, "Esperado '('")
            SE token_atual().tipo != TOKEN_FECHA_PARENTESES:
                args.adicionar(analisar_expressao())
                ENQUANTO token_atual().tipo == TOKEN_VIRGULA:
                    consumir(TOKEN_VIRGULA, "Esperado ','")
                    args.adicionar(analisar_expressao())
            consumir(TOKEN_FECHA_PARENTESES, "Esperado ')'")
        consumir(TOKEN_PONTO_VIRGULA, "Esperado ';'")
        RETORNAR Novo No JoinStmt(args=args)

    METODO analisar_expressao():
        RETORNAR analisar_ou_logico()

    METODO analisar_ou_logico():
        esquerda <- analisar_e_logico()
        ENQUANTO token_atual().tipo == TOKEN_OU_LOGICO:
            op <- consumir(TOKEN_OU_LOGICO, "Esperado '||'").lexema
            direita <- analisar_e_logico()
            esquerda <- Novo No BinaryExpr(op=op, esquerda=esquerda, direita=direita
    )
        RETORNAR esquerda
FIM_CLASSE
```

Listing 4.1 – Pseudocódigo do Analisador Sintático (Parser)

5 ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS

O Analisador Semântico do compilador MiniPar 2026.2, implementado em `src/semantic.py` e operado através do componente `C_Semantico` (interface `ISemantico`), é encarregado de verificar a consistência lógica, estrutural e tipológica da Árvore de Sintaxe Abstrata gerada pelo Parser.

5.1 Estratégia de Duas Passagens (*Two-Pass Analysis*)

Para permitir que funções e classes sejam utilizadas no código-fonte antes de suas declarações textuais (referência antecipada ou *forward declaration*), o analisador adota um modelo de validação em duas etapas:

1. Primeira Passagem (Coleta de Declarações): percorre a lista de comandos do nó raiz `Program` e registra todas as definições globais de classes (com métodos e atributos) e funções (com parâmetros formais) na Tabela de Símbolos. Caso haja duplicação de identificadores no mesmo escopo, um erro semântico é reportado.

2. Segunda Passagem (Validação e Inferência de Tipos): inspeciona cada nó de comando e expressão, validando:

    - Existência e visibilidade de variáveis acessadas em seus respectivos escopos;
    - Compatibilidade estática e coerência em atribuições e operações binárias/unárias;
    - Conformidade da quantidade de argumentos em chamadas de funções e métodos;
    - Validação de contexto de fluxo: controle da profundidade de laços (`loop_depth`), impedindo o uso indevido de `break` ou `continue` fora de blocos de repetição;
    - Uso de `this` restrito ao escopo interno de métodos de classes;
    - Tipagem de primitivas concorrentes (`fork` gera o tipo de tarefa `task`, e canais `c_channel` operam com strings/números).

5.2 Tratamento de Comentários (#) e Diagnósticos de Erros

O Requisito 5 especifica que o compilador deve permitir comentários via `#`:

- Os comentários são descartados na análise léxica, mantendo os contadores de linha e coluna atualizados.

- Quando o Analisador Semântico detecta uma inconformidade (por exemplo, uso de variável não declarada), ele reporta o erro identificando a linha original do código MiniPar, desconsiderando linhas de comentários.

- A saída do erro é padronizada no formato:

```
[ERRO SEMANTICO] Linha L, Coluna C: Mensagem Explicativa
```

5.3 Pseudocódigo do Analisador Semântico

O Algoritmo 5.1 descreve a lógica e as rotinas de verificação do analisador semântico.

```
CLASSE AnalisadorSemantico:
    ATRIBUTOS:
        tabela_simbolos: TabelaDeSimbolos
        erros: Lista de Erros <- []
        classes_declaradas: Mapa[Nome, MetadadosClasse]
        funcoes_declaradas: Mapa[Nome, MetadadosFuncao]
        profundidade_laco: Inteiro <- 0
        classe_atual: Texto <- NULO

    METODO reportar_erro(mensagem, linha, coluna):
        # Permite identificar erros ignorando comentarios ('#') filtrados pelo Lexer
        erro <- Novo Objeto Erro(fase="semantica", linha=linha, coluna=coluna,
    mensagem=mensagem)
        erros.adicionar(erro)
        IMPRIMIR("[ERRO SEMANTICO] Linha " + linha + ": " + mensagem)

    METODO analisar(programa):
        # Passagem 1: Coleta global de classes e funcoes
        PARA CADA declaracao EM programa.comandos:
            SE declaracao eh ClassDecl:
                SE declaracao.nome JA_EXISTE EM classes_declaradas:
                    reportar_erro("Classe '" + declaracao.nome + "' ja declarada",
    declaracao.linha)
                classes_declaradas[declaracao.nome] <- declaracao
            SENAO SE declaracao eh FuncDecl:
                SE declaracao.nome JA_EXISTE EM funcoes_declaradas:
                    reportar_erro("Funcao '" + declaracao.nome + "' ja declarada",
    declaracao.linha)
                funcoes_declaradas[declaracao.nome] <- declaracao

        # Passagem 2: Validacao de escopos e tipos
        PARA CADA comando EM programa.comandos:
            validar_no(comando)

        RETORNAR (tamanho(erros) == 0)

    METODO inferir_tipo(no):
        SE no eh NumberExpr: RETORNAR "number"
        SE no eh StringExpr: RETORNAR "string"
        SE no eh BoolExpr:   RETORNAR "bool"
```

```
    SE no eh ListLiteral: RETORNAR "list"
    SE no eh ForkStmt:   RETORNAR "task"
    SE no eh IdentifierExpr:
        simbolo <- tabela_simbolos.consultar(no.nome)
        SE simbolo != NULO: RETORNAR simbolo.tipo
        RETORNAR "unknown"
    SE no eh BinaryExpr:
        SE no.op EM ["==", "!=", "<", "<=", ">", ">=", "&&", "||"]:
            RETORNAR "bool"
        tipo_esq <- inferir_tipo(no.esquerda)
        tipo_dir <- inferir_tipo(no.direita)
        SE no.op == "+" E (tipo_esq == "string" OU tipo_dir == "string"):
            RETORNAR "string"
        RETORNAR "number"
    RETORNAR "unknown"

METODO validar_no(no):
    SE no eh VarDecl:
        SE no.valor_inicial != NULO:
            validar_no(no.valor_inicial)
            tipo_valor <- inferir_tipo(no.valor_inicial)
            SE no.tipo != "var" E tipo_valor != "unknown" E no.tipo !=
tipo_valor:
                reportar_erro("Tipo incompativel na inicializacao de '" + no.
nome + "'", no.linha)
        declarou <- tabela_simbolos.declarar(no.nome, no.tipo, no.linha)
        SE NAO declarou:
            reportar_erro("Variavel '" + no.nome + "' ja declarada no escopo",
no.linha)

    SENAO SE no eh Assignment:
        validar_no(no.valor)
        SE no.alvo eh IdentifierExpr:
            simbolo <- tabela_simbolos.consultar(no.alvo.nome)
            SE simbolo == NULO:
                reportar_erro("Variavel '" + no.alvo.nome + "' nao declarada",
no.linha)
            SENAO:
                tipo_valor <- inferir_tipo(no.valor)
                SE simbolo.tipo != "var" E tipo_valor != "unknown" E simbolo.
tipo != tipo_valor:
                    reportar_erro("Atribuicao de tipo incompativel em '" + no.
alvo.nome + "'", no.linha)

    SENAO SE no eh WhileStmt OU no eh DoWhileStmt:
        validar_no(no.condicao)
        profundidade_laco <- profundidade_laco + 1
```

```
            tabela_simbolos.entrar_escopo()
            validar_no(no.corpo)
            tabela_simbolos.sair_escopo()
            profundidade_laco <- profundidade_laco - 1

        SENAO SE no eh BreakStmt OU no eh ContinueStmt:
            SE profundidade_laco == 0:
                reportar_erro("Comando break/continue fora de laco de repeticao", no
    .linha)

        SENAO SE no eh SeqBlock OU no eh ParBlock:
            tabela_simbolos.entrar_escopo()
            PARA CADA subcomando EM no.comandos:
                validar_no(subcomando)
            tabela_simbolos.sair_escopo()

        SENAO SE no eh ForkStmt:
            validar_no(no.destino.args)
            validar_no(no.corpo)

        SENAO SE no eh JoinStmt:
            PARA CADA arg EM no.args:
                validar_no(arg)
FIM_CLASSE
```

Listing 5.1 – Pseudocódigo do Analisador Semântico e Tratamento de Erros

Nota de implementação. A análise de par cria um escopo por tarefa. Os diagnósticos usam as posições originais, incluindo as linhas ocupadas por comentários.

6 TABELA DE SÍMBOLOS

A Tabela de Símbolos é a estrutura de dados central encarregada de registrar e consultar identificadores de variáveis, funções, classes, métodos e atributos declarados ao longo da compilação de um programa em MiniPar 2026.2.

No compilador, a gestão de símbolos está formalizada no componente `C_TabelaSimbolos` (`src/componente_tabela_simbolos.py`), que realiza os contratos da interface `ITabelaSimbolos`.

6.1 Estrutura de Dados e Pilha de Escopos Léxicos

A arquitetura adota uma Pilha de Dicionários de Símbolos (*Scope Stack*):

- Escopo Global: corresponde à base da pilha (índice 0), contendo as declarações acessíveis universalmente em todo o módulo compilado.
- Escopos Locais Aninhados: a cada entrada em um novo contexto delimitado (corpo de funções, métodos de classes, blocos de controle `if`, laços `while`/`for` e blocos concorrentes `seq`/`par`), uma nova tabela local é empilhada através da operação `entrar_escopo()`.
- Resolução de Identificadores (Sombra / *Shadowing*): a consulta (`consultar(nome)`) percorre a pilha em sentido descendente (do topo atual para a base). Se um símbolo existir em um escopo mais interno, ele tem prioridade sobre definições em escopos externos, modelando o sombreamento de variáveis.
- Encerramento de Escopo: ao atingir o término de um bloco, o método `sair_escopo()` desempilha a tabela local. Para alimentar relatórios didáticos na interface Web, o componente preserva um histórico de todos os símbolos registrados na sessão.

Cada símbolo é modelado como uma instância da classe `Symbol`, registrando:

- `name`: Nome textual do identificador;
- `type`: Tipo associado (`int`, `number`, `string`, `bool`, `list`, `task`, nome da classe ou `var`);
- `params`: Lista de tipos/nomes de parâmetros (no caso de funções e métodos);
- `attributes`: Dicionário de atributos de instância com seus respectivos tipos;
- `methods`: Dicionário contendo as assinaturas de métodos declarados na classe.

6.2 Pseudocódigo da Tabela de Símbolos

O Algoritmo 6.1 formaliza a implementação do componente de Tabela de Símbolos, atendendo ao Requisito 6.

```
CLASSE Simbolo:
    ATRIBUTOS:
        nome: Texto
        tipo: Texto
        parametros: Lista de Textos <- []
        atributos: Dicionario[Texto, Texto] <- {}
        metodos: Dicionario[Texto, Lista de Textos] <- {}

CLASSE TabelaDeSimbolos:
    ATRIBUTOS:
        pilha_escopos: Pilha de Dicionarios[Texto, Simbolo]
        historico_declaracoes: Lista de Metadados
        proximo_id_escopo: Inteiro <- 1

    METODO inicializar():
        pilha_escopos <- Nova_Pilha()
        escopo_global <- Novo_Dicionario()
        pilha_escopos.empilhar(escopo_global)

    METODO entrar_escopo():
        novo_escopo_local <- Novo_Dicionario()
        pilha_escopos.empilhar(novo_escopo_local)
        proximo_id_escopo <- proximo_id_escopo + 1

    METODO sair_escopo():
        SE tamanho(pilha_escopos) > 1:
            pilha_escopos.desempilhar()

    METODO declarar(nome, tipo, linha=NULO, params=[], attrs={}, methods={}):
        escopo_atual <- pilha_escopos.topo()

        # Impede redeclaracao de identificador no mesmo escopo
        SE nome EXISTE_EM escopo_atual:
            RETORNAR FALSO  # Redeclaracao invalida detectada

        novo_simbolo <- Novo Simbolo(nome=nome, tipo=tipo, parametros=params,
                                     atributos=attrs, metodos=methods)
        escopo_atual[nome] <- novo_simbolo

        # Registra no historico cumulativo para exibicao didatica
        registro <- Novo_Registro(nome=nome, tipo=tipo, linha=linha,
                                  nivel_escopo=tamanho(pilha_escopos) - 1)
```

```
        historico_declaracoes.adicionar(registro)
        RETORNAR VERDADEIRO

    METODO consultar(nome):
        # Percorre a pilha do escopo mais interno (topo) para o global (base)
        PARA i DE (tamanho(pilha_escopos) - 1) ATE 0 PASSO -1:
            escopo <- pilha_escopos[i]
            SE nome EXISTE_EM escopo:
                RETORNAR escopo[nome]
        RETORNAR NULO  # Simbolo nao encontrado em nenhum escopo ativo

    METODO localizar_escopo(nome):
        PARA i DE (tamanho(pilha_escopos) - 1) ATE 0 PASSO -1:
            SE nome EXISTE_EM pilha_escopos[i]:
                RETORNAR i  # 0 indica global, >= 1 indica local
        RETORNAR NULO

    METODO listar_simbolos_historico():
        RETORNAR historico_declaracoes
FIM_CLASSE
```

Listing 6.1 – Pseudocódigo da Tabela de Símbolos

7 MOTOR DE OTIMIZAÇÃO NO BACKEND

Foi integrado ao compilador um Motor de Otimização Independente de Máquina, atuando no backend diretamente sobre a Árvore de Sintaxe Abstrata (AST) antes da geração de código.

O módulo está implementado em `src/optimizer.py` e empacotado no componente `C_Otimizador` (`src/componente_otimizador.py`), obedecendo à interface `IOtimizador`.

7.1 Técnicas de Otimização Implementadas

A classe `ASTOptimizer` implementa transformações semânticas que preservam o comportamento observável do programa, reduzindo a profundidade da árvore sintática, eliminando nós supérfluos e computando valores constantes em tempo de compilação:

1. Dobra de Constantes (*Constant Folding*): expressões aritméticas, lógicas e relacionais formadas exclusivamente por literais estáticos são avaliadas previamente pelo compilador, substituindo subárvores inteiras por um nó literal único:
   - $2 + 3 \times 4 \implies 14$
   - `"Compiladores "` + `"2026.2"` $\implies$ `"Compiladores 2026.2"`
   - `true` && `false` $\implies$ `false`
2. Eliminação de Código Morto (*Dead Code Elimination*): comandos e blocos inalcançáveis em tempo de execução são podados da AST:
   - Blocos `if (false) { ... } else { B }` têm o teste condicional e o ramo principal descartados, retendo-se diretamente o ramo `B`.
   - Blocos `if (true) { A } else { ... }` são simplificados diretamente para o corpo `A`.

A versão examinada aplica dobra de constantes e simplificação de ramos constantes. Identidades algébricas com variáveis foram retiradas.

7.2 Ponto Fixo
As passadas terminam quando deixam de ocorrer transformações ou atingem o limite de dez iterações. Corpos de funções e métodos com retorno são preservados.

A otimização pode ser habilitada opcionalmente:

- Linha de Comando (CLI): através da flag `-O` ou `-optimize`:

```
python main.py tests/oficiais/teste02_fatorial_fibonacci_par.minipar -O
```

- Interface Web Didática: através do seletor *"Otimizar AST"*, que exibe um relatório com o número de nós antes e depois da transformação, o percentual de redução obtido e o registro textual de todas as regras aplicadas.

8 GERAÇÃO DE CÓDIGO INTERMEDIÁRIO (TAC)

O Código Intermediário de Três Endereços (*Three-Address Code* – TAC) é a representação intermediária linear de baixo nível gerada a partir da Árvore de Sintaxe Abstrata (AST), aproximando a semântica da linguagem MiniPar das linguagens de montagem (Assembly) e facilitando fases posteriores de análise e emissão de código de máquina.

No compilador MiniPar 2026.2, o gerador de TAC está implementado em `src/tac.py` e encapsulado no componente `C_TAC` (`src/componente_tac.py`), satisfazendo aos contratos da interface `ITAC`.

8.1 Estrutura do TAC e Linearização de Controle

Cada instrução de três endereços possui, no máximo, um operador no lado direito e até três operandos (dois de entrada e um de destino):

- Registradores Temporários: expressões complexas são decompostas recursivamente por meio da alocação de variáveis temporárias com prefixo `t` (`t0`, `t1`, `t2`, ...).
- Rótulos e Saltos de Fluxo: estruturas de controle e laços (`if`, `while`, `for`) são linearizadas em desvios condicionais (`ifFalse cond goto LABEL`) e incondicionais (`goto LABEL`).
- Mapeamento de Paralelismo (`fork` e `join`): a emissão de TAC suporta as diretivas de paralelismo:

```
t0 = FORK FORK_START_0, core 2
goto ENDFORK_1
FORK_START_0:
# Corpo da subtarefa alocada ao nucleo 2
...
ENDFORK_1:
JOIN t0
```

8.2 Verificação do TAC no Teste 7 (Quicksort Iterativo)

O Teste 7 verifica a ordenação e a geração do TAC. A comparação com a referência em modo verboso é informativa.

O Listing 8.1 reproduz o TAC de referência fornecido pelo professor.
Após o Listing 8.1, um trecho separado mostra a saída real do componente C_TAC.

```
t0 = 0
t1 = 4
S[0] = t0
S[1] = t1
sp = 2

LOOP:
if sp == 0 goto END
sp = sp - 1
t2 = S[sp]
sp = sp - 1
t3 = S[sp]
if t3 >= t2 goto LOOP

t4 = A[t2]
t5 = t3 - 1
t6 = t3

PARTITION_LOOP:
if t6 >= t2 goto PARTITION_END
t7 = A[t6]
if t7 <= t4 goto SWAP
t6 = t6 + 1
goto PARTITION_LOOP

SWAP:
t5 = t5 + 1
t8 = A[t5]
A[t5] = A[t6]
A[t6] = t8
t6 = t6 + 1
goto PARTITION_LOOP

PARTITION_END:
t5 = t5 + 1
t9 = A[t5]
A[t5] = A[t2]
A[t2] = t9

t10 = t5 - 1
S[sp] = t3
sp = sp + 1
S[sp] = t10
sp = sp + 1

t11 = t5 + 1
```

```
S[sp] = t11
sp = sp + 1
S[sp] = t2
sp = sp + 1
goto LOOP

END:
# Array A[0..4] devidamente ordenado
```

Listing 8.1 - TAC de referência para o Quicksort Iterativo (Teste 7)

Trecho do TAC gerado pelo compilador para o Teste 7

```
func quicksort_iterativo:
t0 = [0, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
stack = t0
sp = 2
WHILE_0:
t1 = sp > 0
ifFalse t1 goto ENDWHILE_1
t2 = sp - 1
sp = t2
```

9 GERAÇÃO DE CÓDIGO C++20 E RUNTIME NATIVA

A etapa final do pipeline de compilação (*Back-End*) converte a Árvore de Sintaxe Abstrata validada e otimizada em código-fonte C++ nativo, em conformidade com o padrão ISO C++20.

Essa translação está implementada em `src/codegen.py` e empacotada no componente `C_Codegen` (`src/componente_codegen.py`), que realiza a interface `ICodegen`.

9.1 Transpilação da AST para C++20

A geração de código traduz sistematicamente cada construção sintática do MiniPar 2026.2 para equivalentes semânticos em C++20:

- Orientação a Objetos: classes são convertidas em `class` C++ com membros públicos. Instanciações via `new` utilizam ponteiros inteligentes de contagem de referência (`std::shared_ptr` via `std::make_shared`), com gerenciamento automático de memória. Chamadas de métodos e acessos a atributos utilizam o operador seta (`->`).
- Dedução Estática de Tipos: utiliza a palavra-chave `auto` e referências universais (`auto&&`) nos parâmetros de funções geradas, permitindo flexibilidade sem comprometer o desempenho estático.
- Coleções e Matrizes: mapeadas para `std::vector`, dispondo de funções auxiliares nativas para fatiamento, concatenação e instanciação multidimensional (`make_matrix`).

A Tabela 3 ilustra as correspondências entre o código MiniPar e o C++20 gerado.

Tabela 3 – Mapeamento de Transpilação MiniPar para C++20

Construção em MiniPar 2026.2	Código C++20 Gerado
`class P { var x; func m() {} }`	`class P { public: auto x; auto m() {} };`
`var obj = new P();`	`auto obj = std::make_shared<P>();`
`obj.m();`	`obj->m();`
`par { cmd1; cmd2; }`	`std::vector<std::thread> __thr; __thr.emplace_back([=]() mutable { cmd1; }); for(auto& t : __thr) t.join();`
`fork core(2) { tarefa(); }`	`__register_task(std::async(std::launch::async, [=]{ __set_core_affinity(2); return tarefa(); }));`
`join;`	`__task_pool_wait_all();`
`c_channel("127.0.0.1", 5000)`	`std::make_shared<MiniParChannel>("127.0.0.1", 5000)`

9.2 Runtime Nativa e Afinidade de Núcleos (*Core Affinity*)

O arquivo `src/cpp_template.py` atua como cabeçalho de suporte em tempo de execução (*Runtime Engine*), sendo embutido no preâmbulo de todos os arquivos C++ gerados. Seus principais módulos compreendem:

9.2.1 Isolamento de Núcleos de CPU (Afinidade de Hardware)

A afinidade solicita que o sistema operacional execute a thread no processador lógico indicado.

O runtime ajusta o índice ao total detectado. A API pode recusar a solicitação.

A afinidade restringe onde a thread pode rodar, sem reservar o núcleo.

- Ambiente Windows: invoca `SetThreadAffinityMask(GetCurrentThread(), mask)`, definindo uma máscara binária de afinidade que força a thread a executar exclusivamente no núcleo selecionado.
- Ambiente Linux / POSIX: invoca `pthread_setaffinity_np` associado a estruturas `cpu_set_t`.

9.2.2 Pool de Tarefas e Barreiras Assíncronas

Tarefas disparadas pelo comando `fork` são encapsuladas em `std::shared_future` através da função auxiliar `__register_task`. Essas tarefas são registradas em uma lista com escopo de thread local (`thread_local std::vector<std::function<void()» __task_pool`).

Quando uma barreira `join;` é encontrada no código MiniPar, o runtime invoca `__task_pool_wait_all()`, garantindo que todas as tarefas pendentes na fila concluam de forma sincronizada antes do avanço do programa. Quando o comando especifica argumentos particulares (`join(t1, t2)`), o helper `__minipar_join_target` bloqueia seletivamente as instâncias correspondentes.

A lista guarda futuros para sincronização. Cada fork usa std::async para iniciar a tarefa.

9.2.3 Comunicação TCP com Resiliência

A classe `MiniParChannel` implementa comunicação cliente/servidor baseada em sockets TCP:

- Utiliza a biblioteca `ws2_32.lib` no Windows (inicializada via `WSAStartup`) e os cabeçalhos padrão `sys/socket.h` e `arpa/inet.h` em sistemas Unix.
- A operação `send` implementa um laço de tentativas de conexão com recriação de descritores de socket, tolerando atrasos de inicialização do servidor sem abortar a execução.
- A operação `receive` configura um socket servidor (`bind`, `listen` e `accept`), aguarda o pacote de dados do cliente, extrai a mensagem e encerra a conexão de forma limpa.

9.3 Compilação Nativa com g++

Concluída a translação da AST para o arquivo `output/<nome>.cpp`, o compilador invoca automaticamente o compilador nativo `g++` através do módulo `subprocess` do Python:

```
g++ -std=c++20 -O3 output/<nome>.cpp -o output/<nome>.exe -pthread -lws2_32
```

A flag `-std=c++20` ativa o suporte às diretivas modernas de concorrência, enquanto `-O3` aciona as otimizações agressivas de vetorização, desenrolamento de laços (*loop unrolling*) e inlining do GCC, gerando um binário executável final de alto desempenho.

10 INTERFACES DO COMPILADOR (3 MODALIDADES)

Conforme especificado no enunciado oficial do projeto MiniPar 2026.2, o sistema oferece três modalidades distintas de interface: Linha de Comando (CLI), Primeira Interface Web (Desenvolvedor) e Segunda Interface Web (Didática).

10.1 Modalidade 1: Interface de Linha de Comando (CLI) e Menu Terminal

A interface por linha de comando (`main.py` e `menu.py`) é orientada à automação em esteiras de integração contínua (CI) e à execução ágil de testes diretamente pelo console:

- Modo Direto: permite compilar qualquer arquivo `.minipar` passando seu caminho como argumento:

```
python main.py tests/oficiais/teste07_quicksort_iterativo_tac.minipar
```

- Menu Interativo de Testes: executado via `python menu.py`, exibe uma lista numerada dos 11 testes oficiais, permitindo executar todos os testes em lote ou disparar um teste individual com captura de entrada do usuário.
- Coordenação Multi-Terminal: o módulo `test_orchestrator.py` coordena múltiplos terminais abertos para cenários distribuídos, monitorando o progresso de cada participante através de estado compartilhado em `output/.test_session.json`.

10.2 Modalidade 2: Primeira Interface Web (Desenvolvedor)

A primeira interface Web é voltada ao perfil de desenvolvedor e pode ser acessada na rota:

`http://127.0.0.1:5000/ide`

Essa interface apresenta:

- Editor de código baseado no CodeMirror com realce de sintaxe;
- Painel lateral com lista de arquivos de testes;
- Abas superiores com visualização simultânea do código C++ gerado e da saída em Três Endereços (TAC);
- Terminal inferior integrado para exibição da saída padrão (`stdout`) e mensagens de erro do compilador.

A Figura 1 ilustra a interface de desenvolvedor, permitindo editar programas MiniPar, compilar com exibição lado a lado de C++ e TAC, e executar com saída no terminal integrado.

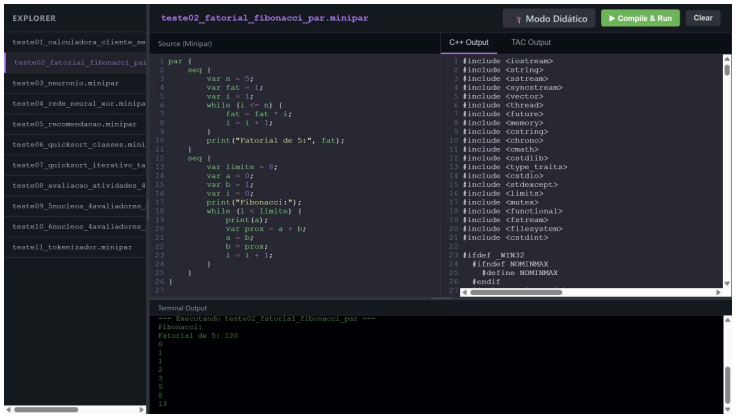


Figura 1 – Primeira Interface Web (IDE do Desenvolvedor – `/ide`)

10.3 Modalidade 3: Segunda Interface Web (Didática / Educacional)

O compilador inclui a Segunda Interface Web Didática, concebida para atender ao aprendizado prático de compiladores. Ela está disponível na rota raiz e didática do servidor:

`http://127.0.0.1:5000/` e `http://127.0.0.1:5000/didatico`

10.3.1 Inspiração Visual e Filosofia de Design

Inspirada no compilador educacional do Instituto Federal de Santa Catarina (IFSC), a interface apresenta um layout limpo, moderno, responsivo e modular, com painéis ajustáveis e controles organizados por fase.

10.3.2 Controles Didáticos Passo a Passo (Botões de Fase)

Diferentemente de compilações monolíticas em que todo o processo ocorre em caixa-preta, a interface didática decompõe o processo de compilação em botões dedicados que podem ser acionados passo a passo:

1. Tokens – Lexer (Botão): apresenta uma tabela interativa contendo todos os tokens identificados pelo analisador léxico, detalhando a categoria sintática (`TokenType`), o lexema original, a linha e a coluna exatas, além de ferramentas de busca e filtros por categoria.

2. AST – Árvore Sintática (Botão): em vez de apenas um dump textual ou ASCII, a interface renderiza uma árvore sintática interativa em SVG/D3.js. Os nós possuem cores temáticas por categoria (comandos, expressões, classes), são colapsáveis com suporte a expansão sob clique, navegação por arrasto (*pan*) e ampliação dinâmica (*zoom*). Árvores extensas iniciam recolhidas para manter a tela limpa, com botão *"Expandir Tudo"* para inspeção completa.

3. Análise Semântica (Botão): exibe o status da validação semântica, lista de diagnósticos com linha/coluna e a Tabela de Símbolos completa, discriminando escopos léxicos, nomes, tipos e assinaturas.

4. TAC – Código de 3 Endereços (Botão): exibe o código intermediário linear destrinchado instrução por instrução com destaque sintático para registradores temporários e rótulos de controle.

5. Código C++ Gerado (Botão): exibe o código-fonte C++20 transpilado gerado em memória pelo componente `ICodegen`.

6. Compilar e Baixar Executável (Botão): dispara a compilação nativa isolada no backend via `g++ -O3` e disponibiliza o download direto do binário executável (`.exe` no Windows ou executável ELF no Linux).

7. Executar (Botão) inicia um processo nativo no servidor,

   com suporte à passagem de dados pré-digitados no painel de *Terminal & Execução* (uma resposta por linha, preservando espaços e vírgulas).

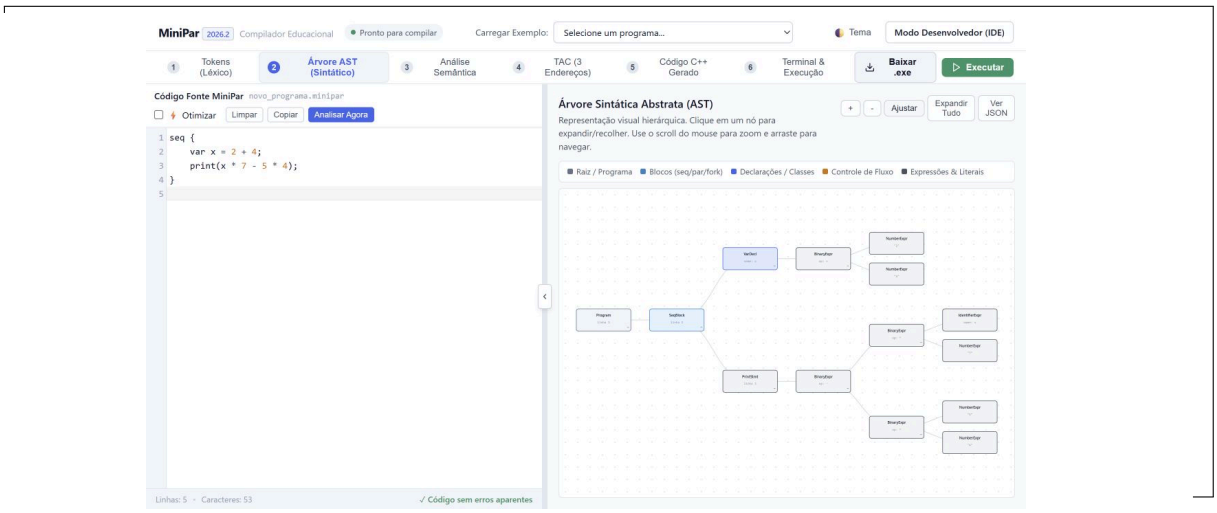


Figura 2 – Segunda Interface Web Didática do MiniPar 2026.2 (Visualizador Gráfico Interativo de AST)

10.3.2 Controles didáticos: análise léxica e semântica

A tabela léxica associa cada token à sua posição no fonte. A análise semântica reúne o resultado da validação e os símbolos por escopo.

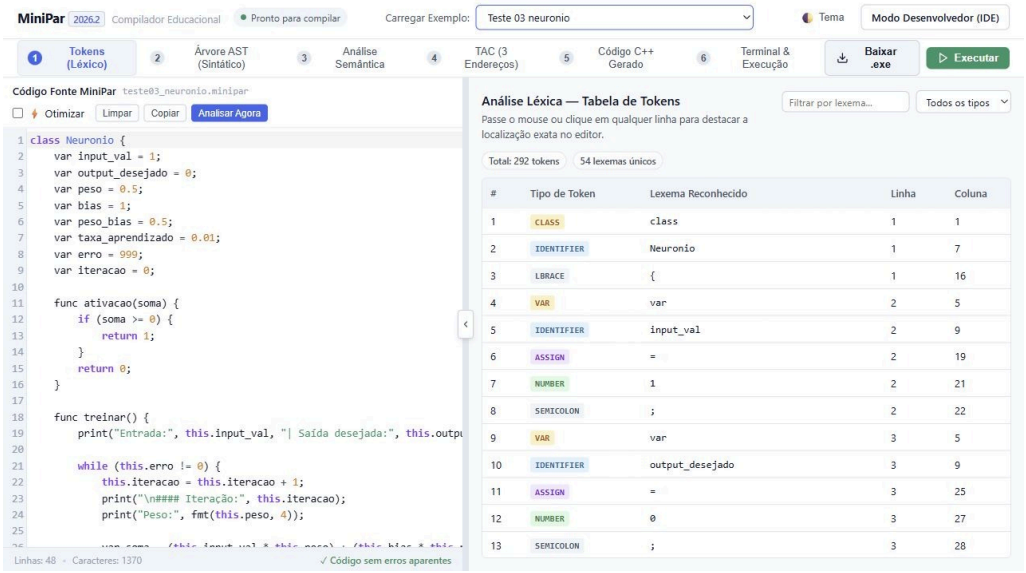


Figura 2A - Tokens e posições no código-fonte.

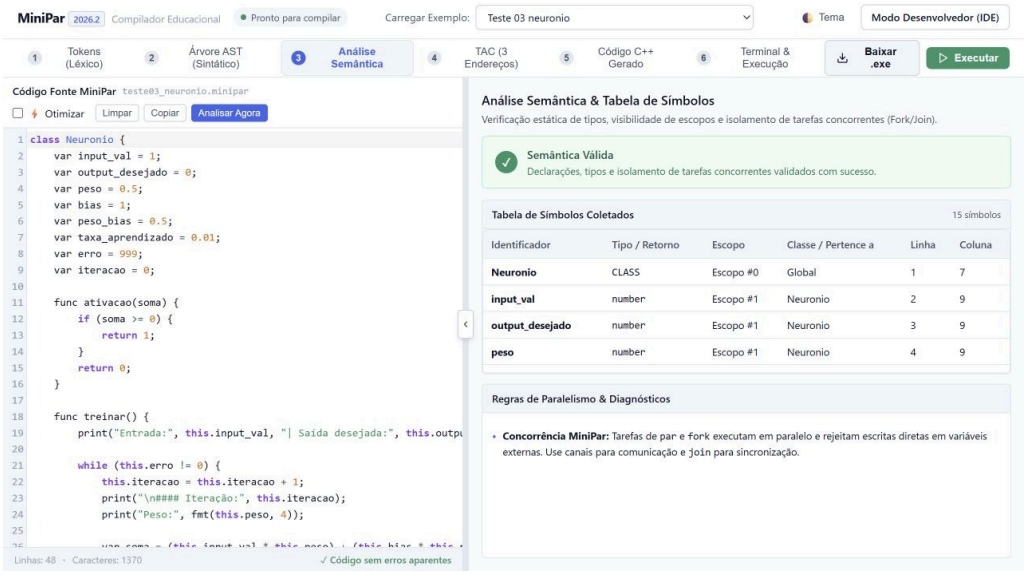


Figura 2B - Análise semântica e tabela de símbolos do Teste 3.

10.3.2 Controles didáticos: diagnóstico e código intermediário

Quando ocorre um erro, o diagnóstico informa a fase e a posição. Em um programa válido, o painel TAC mostra os passos intermediários.

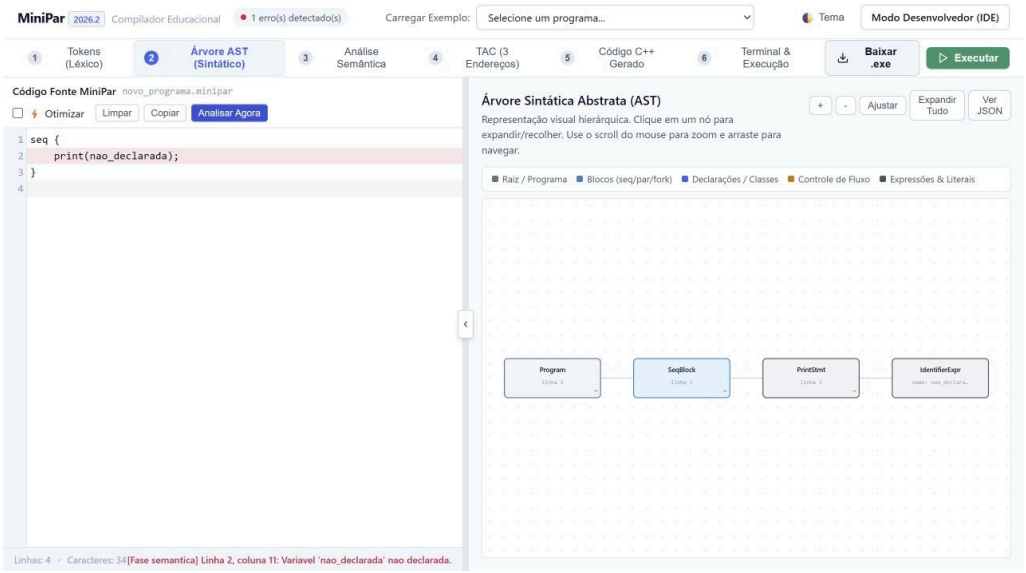


Figura 2C - Diagnóstico semântico com linha e coluna.

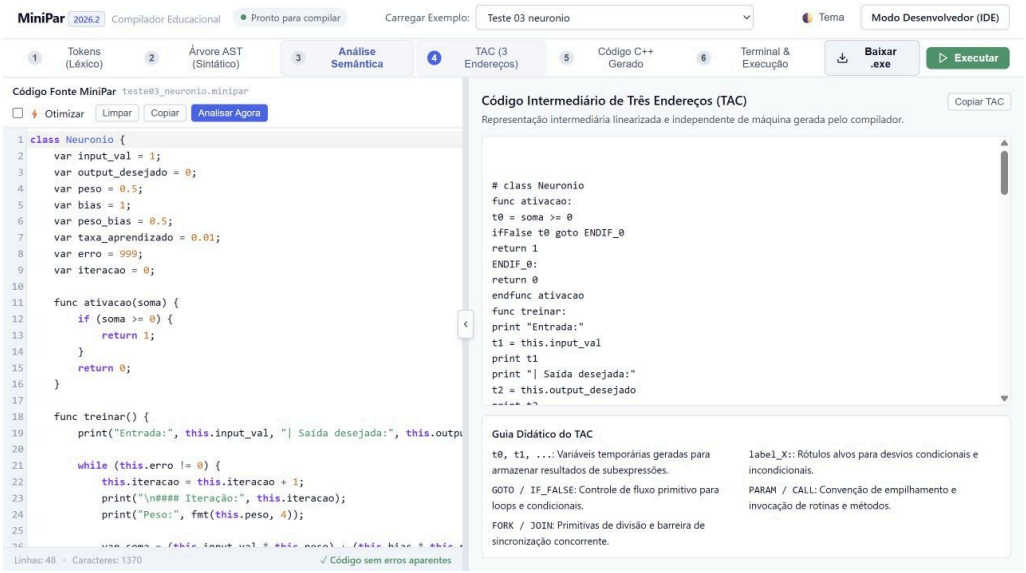


Figura 2D - Código de três endereços gerado para o Teste 3.

10.3.2 Controles didáticos: C++ e executável

A interface permite inspecionar o C++ gerado e baixar o executável compilado a partir do programa no editor.

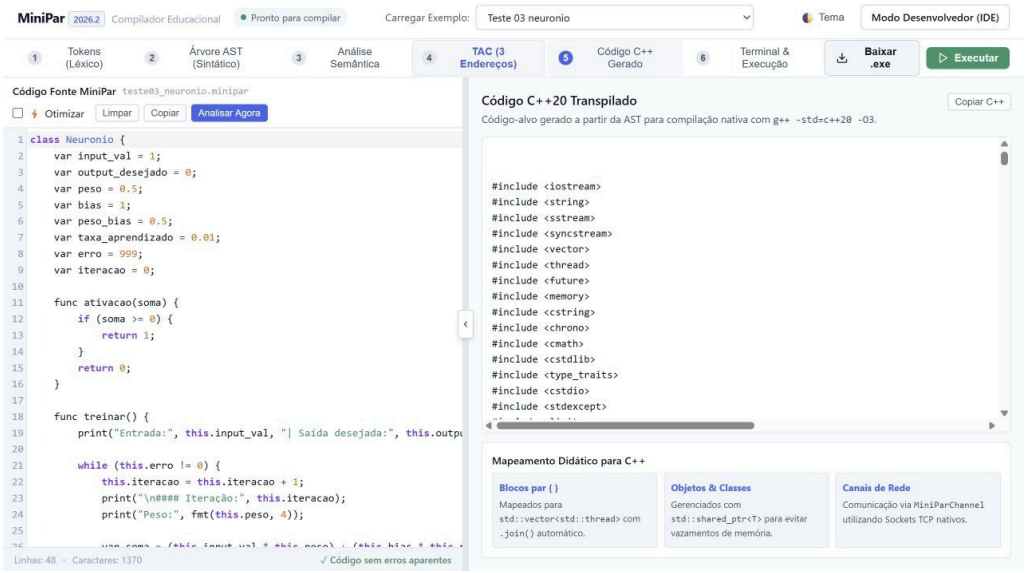


Figura 2E - Visualização do C++ gerado para o Teste 3.

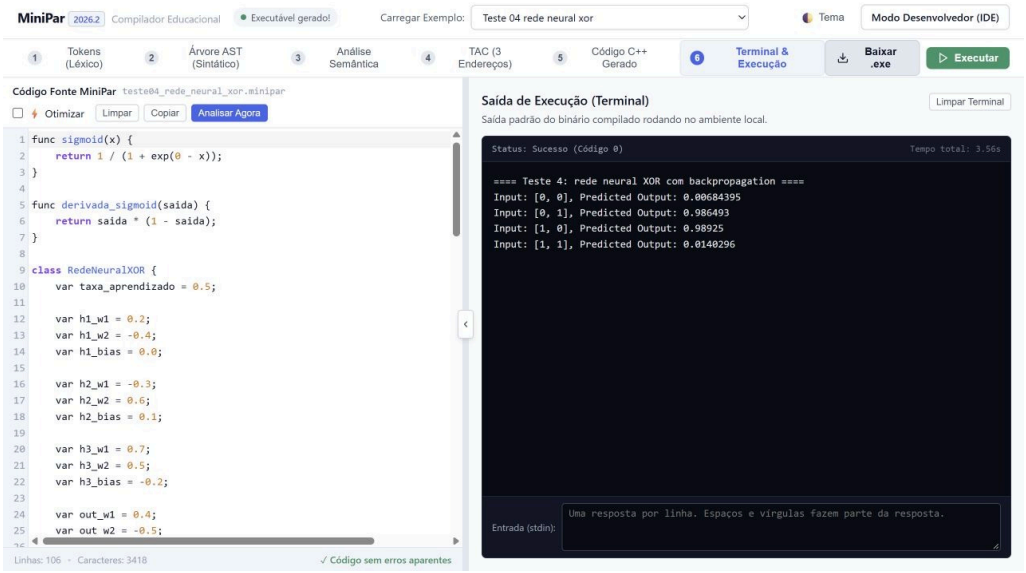


Figura 2F - Download do executável pela interface didática.

11 GERÊNCIA DE PROJETO ÁGIL (PRODUCT E SPRINTS BACKLOG)

O desenvolvimento do compilador MiniPar 2026.2 foi orientado pelo framework ágil Scrum, estruturado em entregas incrementais contínuas (*sprints*) com rastreabilidade formal entre histórias de usuário, requisitos técnicos da disciplina e tarefas de implementação.

11.1 Product Backlog

O Product Backlog consolidado foi definido a partir do mapeamento direto entre as exigências especificadas para o compilador e as funcionalidades efetivamente implementadas pela equipe. A Tabela 4 documenta os itens priorizados, correlacionando o que foi solicitado com a solução técnica entregue.

11.2 Sprints Backlog

O planejamento das atividades foi estruturado em seis *sprints* incrementais de desenvolvimento, organizando os itens do Product Backlog em metas técnicas bem delimitadas, conforme apresentado na Tabela 5.

Tabela 4 – Product Backlog do Compilador MiniPar 2026.2

Item	Descrição (Especificado vs. Implementado)	Prioridade	Status
PB01	Especificação: gramática para real paralelismo com `fork` e `join`. Implementação: extensão da EBNF, novos tokens e nós na AST.	Alta	Concluído
PB02	Especificação: arquitetura por componentes de software (CBSE). Implementação: desacoplamento nos 7 componentes com interfaces formais.	Alta	Concluído
PB03	Especificação: controle de escopos e tipos concorrentes. Implementação: Tabela de Símbolos em pilha, tipo `task` e validação de laços.	Alta	Concluído
PB04	Especificação: suporte a afinidade de núcleos de hardware. Implementação: chamadas Win32/POSIX e pool assíncrono thread-safe na runtime C++.	Alta	Concluído
PB05	Especificação: canais de comunicação por mensagens. Implementação: classe `MiniParChannel` com sockets TCP/IP no Windows.	Alta	Concluído
PB06	Especificação: emissão de TAC com temporários e validação formal. Implementação: linearização em `tac.py` e validação do Teste 7.	Alta	Concluído
PB07	Especificação: algoritmos de multiprocessamento com Speedup e fractais. Implementação: execução aprovada dos Testes 8, 9 e 10 com BMP.	Alta	Concluído
PB08	Especificação: Segunda Interface Web didática estilo IFSC. Implementação: aplicação Web didática com AST interativa em D3.js e botões por fase.	Alta	Concluído
PB09	Especificação: preservação das interfaces herdadas. Implementação: manutenção da CLI direta, menu de testes e rota de desenvolvedor (`/ide`).	Média	Concluído
PB10	Especificação: coordenação de múltiplos terminais em testes. Implementação: módulo `test_orchestrator.py` com sessões compartilhadas.	Média	Concluído
PB11	Especificação: minicompilador de expressões aritméticas com ARMv7. Implementação: Teste 11 em MiniPar com gravação em disco de Assembly ARMv7.	Alta	Concluído
PB12	Especificação: otimização estática independente na AST. Implementação: motor no backend com dobra de constantes e eliminação de código morto.	Média	Concluído
PB13	Especificação: controle opcional de otimização na CLI e Web. Implementação: flag `-O` no compilador e opção de otimização na interface didática.	Média	Concluído
PB14	Especificação: garantia de qualidade e conformidade UML. Implementação: 135 testes aprovados e diagramas UML.	Alta	Concluído

Tabela 5 – Sprints Backlog do Compilador MiniPar 2026.2

Sprint	ID	Descrição Técnica da Tarefa	Ref. PB
Sprint 1	1.1	Extensão da gramática BNF e adição dos tokens `FORK`, `JOIN`, `CORE` e `HOST`.	PB01
	1.2	Modelagem dos nós `ForkStmt`, `JoinStmt` e `Destino` na AST.	PB01
	1.3	Implementação inicial de afinidade de CPU e template de suporte C++.	PB04
	1.4	Transpilação preliminar de comandos `fork` e `join` para C++ e TAC.	PB01, PB06
Sprint 2	2.1	Desacoplamento da arquitetura nos componentes CBSE (`ILexico`, `ISintatico`, etc.).	PB02
	2.2	Extração e desacoplamento do componente de Tabela de Símbolos.	PB02, PB03
	2.3	Isolamento de escopo concorrente com `thread_local` e captura em blocos paralelos.	PB03, PB04
	2.4	Implementação de pool de tarefas thread-safe com barreira de sincronização.	PB04
Sprint 3	3.1	Implementação e validação do Teste 8 (160 avaliações com Speedup em 4 núcleos).	PB07
	3.2	Implementação do Teste 9 (avaliação paralela e renderização de Mandelbrot ASCII).	PB07
	3.3	Implementação do Teste 10 (multiprocessamento em 6 núcleos com Sierpinski em BMP).	PB07
	3.4	Verificação da geração de TAC no Quicksort Iterativo (Teste 7).	PB06
Sprint 4	4.1	Desenvolvimento da interface Web didática com árvore sintática em SVG/D3.js.	PB08
	4.2	Integração dos botões de controle de compilação passo a passo e rotas Flask.	PB08, PB09
	4.3	Resiliência a reconexões de sockets TCP/IP no Windows para canais `c_channel`.	PB05
	4.4	Implementação do orquestrador multi-terminal e isolamento dos 11 testes oficiais.	PB09, PB10
Sprint 5	5.1	Desenvolvimento do minicompilador de expressões aritméticas no Teste 11.	PB11
	5.2	Adição da função nativa de gravação em disco (`write_file`) na runtime C++.	PB11
	5.3	Síntese e gravação de código Assembly ARMv7 validado no simulador CPULator.	PB11
	5.4	Integração do Teste 11 no executor automatizado de testes de regressão.	PB11, PB14
Sprint 6	6.1	Criação da interface `IOtimizador` e componente `C_Otimizador` no backend.	PB12
	6.2	Implementação de dobra de constantes e código morto.	PB12
	6.3	Integração da flag `-O` na CLI e opções de otimização na interface Web.	PB13
	6.4	Execução da suíte de testes automatizados no pytest e documentação UML.	PB14

12 MODELAGEM UML BASEADA EM COMPONENTES DE SOFTWARE (CBSE)

O MiniPar 2026.2 usa componentes com interfaces Python para as fases de compilação.
O capítulo apresenta os casos de uso, as dependências entre os módulos e as classes
que implementam cada interface.

12.1 Diagrama de Casos de Uso

O Diagrama de Casos de Uso da Figura 3 modela as interações do usuário e desenvolvedor com as funcionalidades do compilador MiniPar 2026.2.

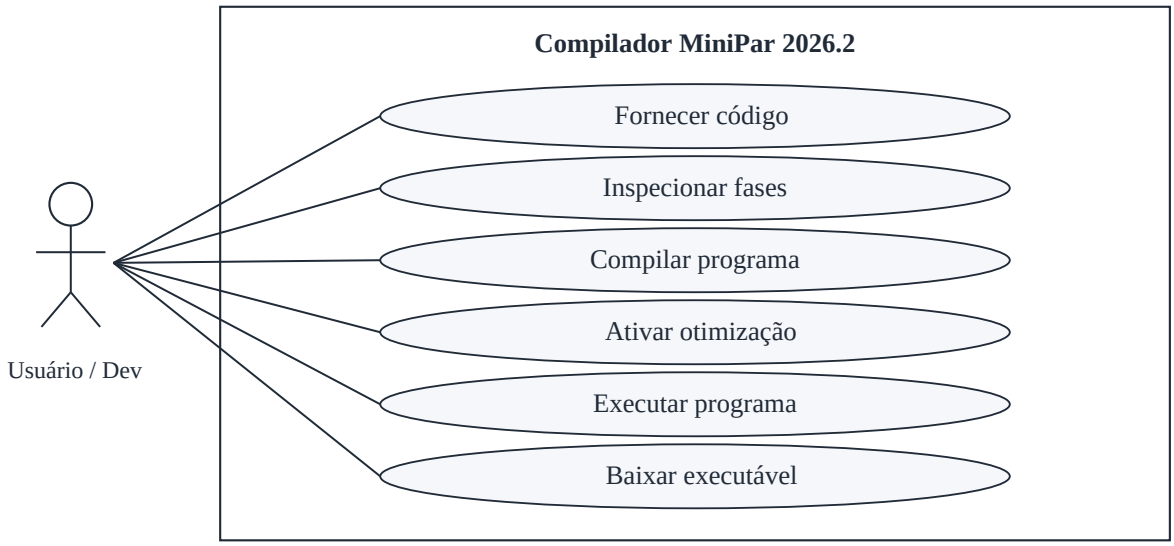


Figura 3 – Diagrama de Casos de Uso segundo a notação da UML

12.2 Arquitetura utilizando Componentes de Software

A Figura 4 mostra as dependências entre os módulos de controle e os componentes.
As interfaces indicadas nos blocos são os contratos fornecidos por cada componente.

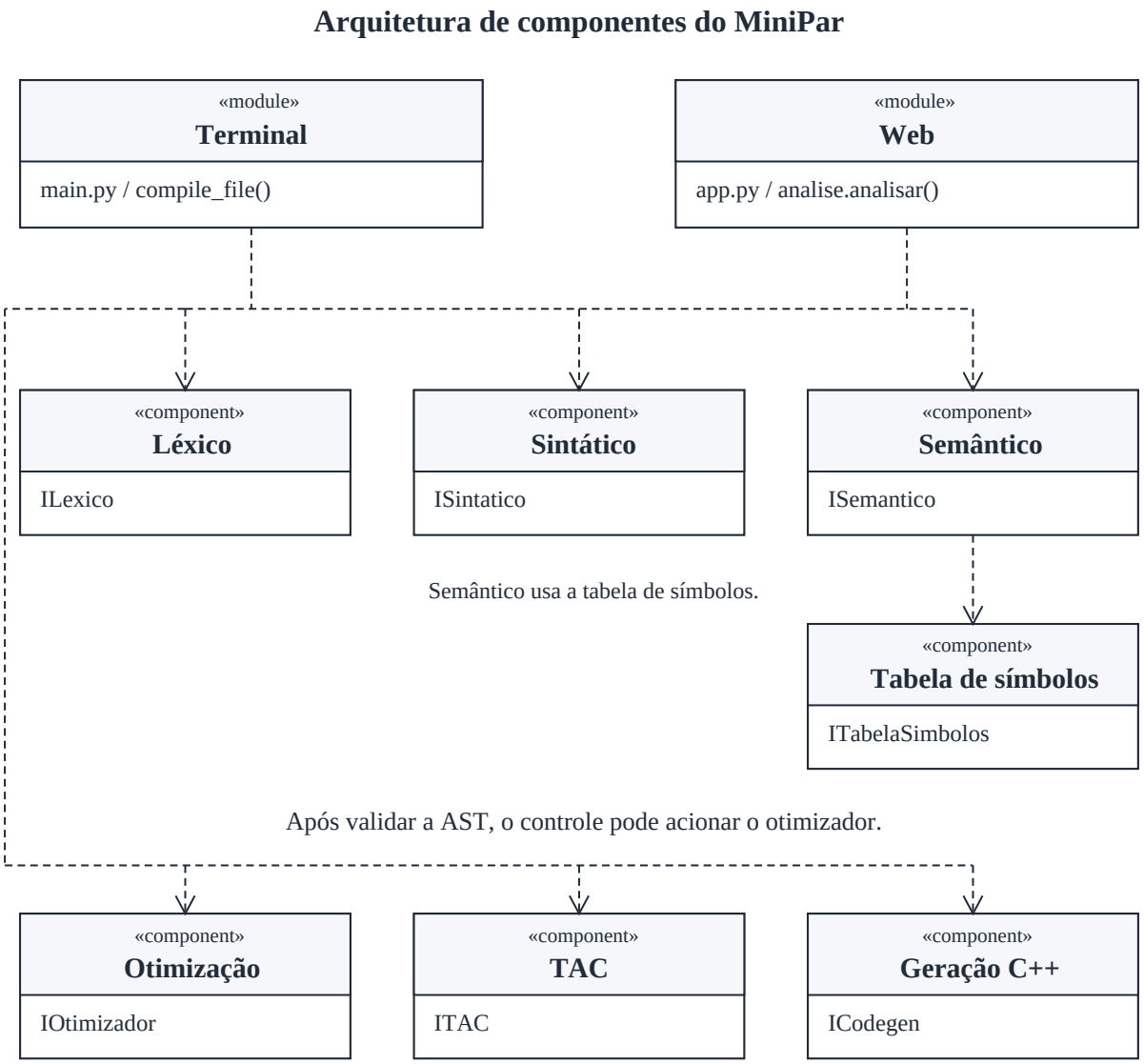


Figura 4 – Arquitetura utilizando Componentes de Software segundo a notação da UML

12.3 Modelagem dos Componentes de Software via Diagrama de Classes

Os diagramas mostram as interfaces e as classes que as implementam.
Ao final, os módulos de controle aparecem com os pontos de entrada do código.

12.3.1 Componente Analisador Léxico

A Figura 5 apresenta a modelagem do analisador léxico, composto pela interface `ILexico` e pela classe componente `C_Lexico`.

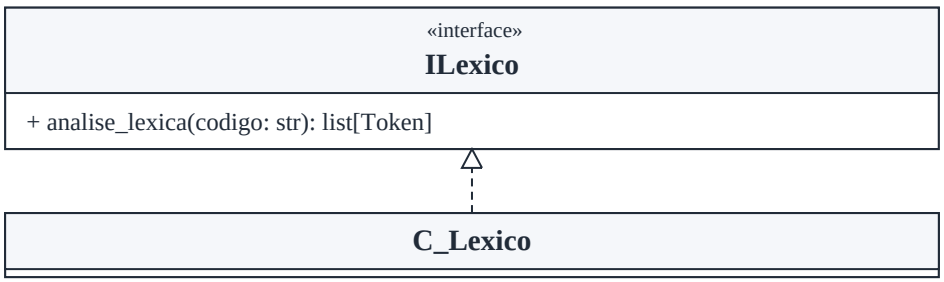


Figura 5 – Diagrama de Classes do Componente Analisador Léxico (`ILexico` e `C_Lexico`)

12.3.2 Componente Analisador Sintático

A Figura 6 ilustra o analisador sintático, formado pela interface `ISintatica` e pela classe componente `C_Sintatico`.

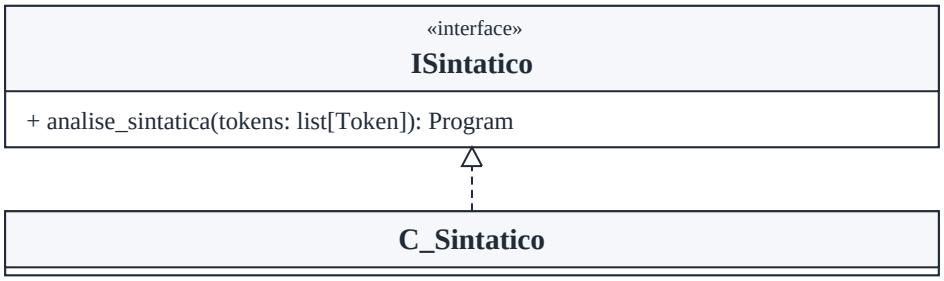


Figura 6 – Diagrama de Classes do Componente Analisador Sintático (`ISintatica` e `C_Sintatico`)

12.3.3 Componente Tabela de Símbolos

A Figura 7 modela o gerenciador de escopos e símbolos, composto pela interface `ITabelaSimbolos` e pela classe `C_TabelaSimbolos`.

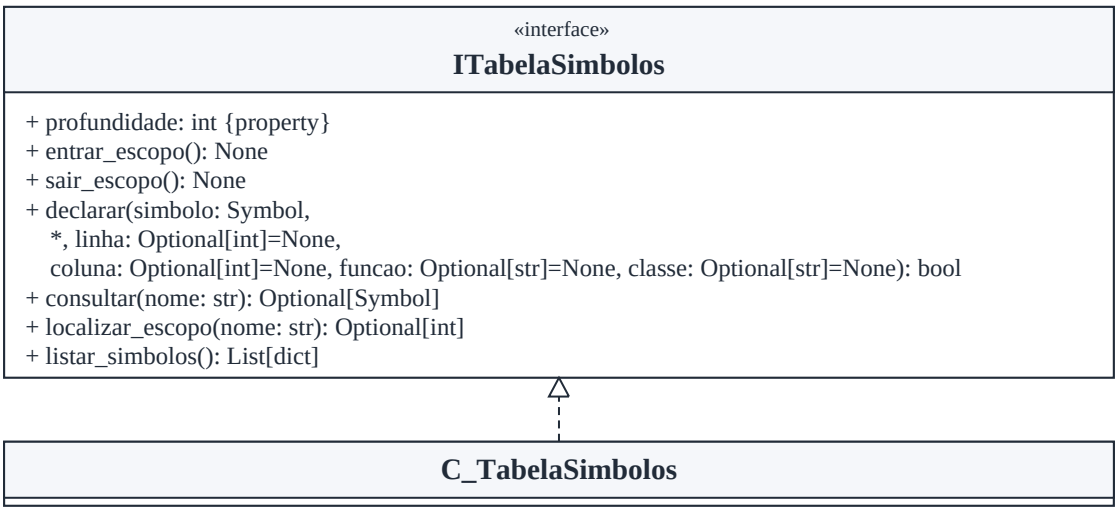


Figura 7 – Diagrama de Classes do Componente Tabela de Símbolos (`ITabelaSimbolos` e `C_TabelaSimbolos`)

12.3.4 Componente Analisador Semântico

A Figura 8 detalha o analisador semântico e checador de tipos, formado pela interface `ISemantica` e pela classe `C_Semantico`.

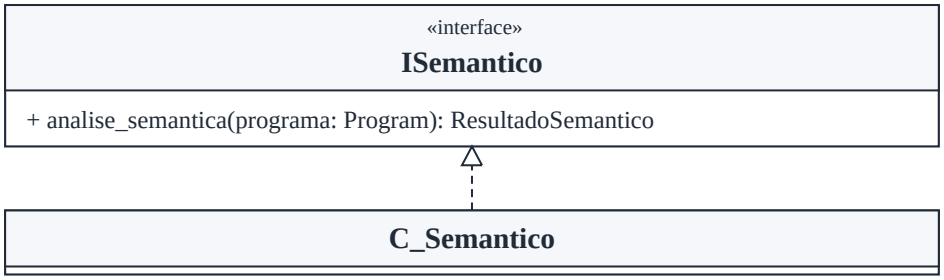


Figura 8 – Diagrama de Classes do Componente Analisador Semântico (`ISemantica` e `C_Semantico`)

12.3.5 Componente Otimizador

A Figura 9 ilustra o motor de otimização da AST, composto pela interface `IOtimizador` e pela classe componente `C_Otimizador`.

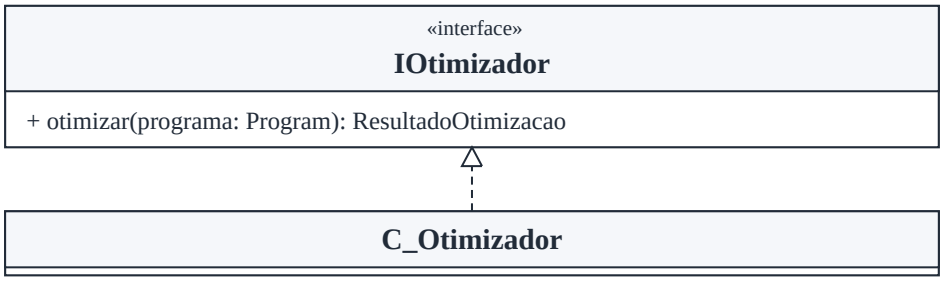


Figura 9 – Diagrama de Classes do Componente Otimizador (`IOtimizador` e `C_Otimizador`)

12.3.6 Componente Gerador de Código Intermediário (TAC)

A Figura 10 modela a geração de instruções de Três Endereços, composta pela interface `ITAC` e pela classe componente `C_TAC`.

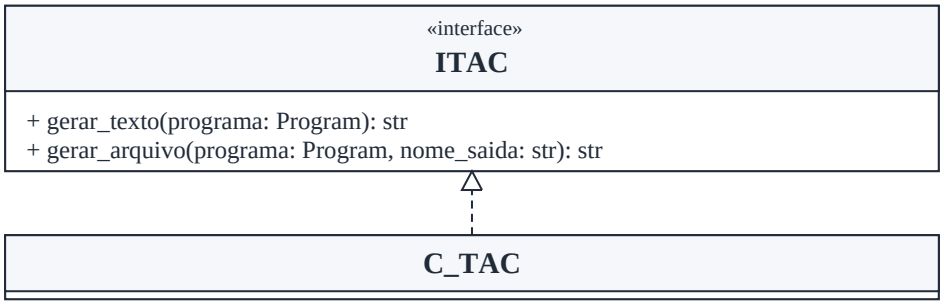


Figura 10 – Diagrama de Classes do Componente Gerador de TAC (`ITAC` e `C_TAC`)

12.3.7 Componente Gerador de Código C++

A Figura 11 apresenta o gerador de código C++20 nativo, estruturado pela interface `ICodegen` e pela classe `C_Codegen`.

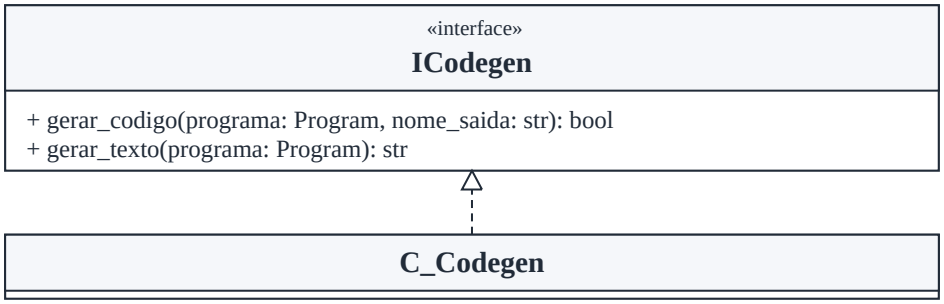


Figura 11 – Diagrama de Classes do Componente Gerador de Código C++ (`ICodegen` e `C_Codegen`)

12.3.8 Controle e Interfaces

A Figura 12 mostra os módulos que coordenam a compilação no terminal e na Web.
O controle é implementado por funções em main.py, app.py e analise.py.

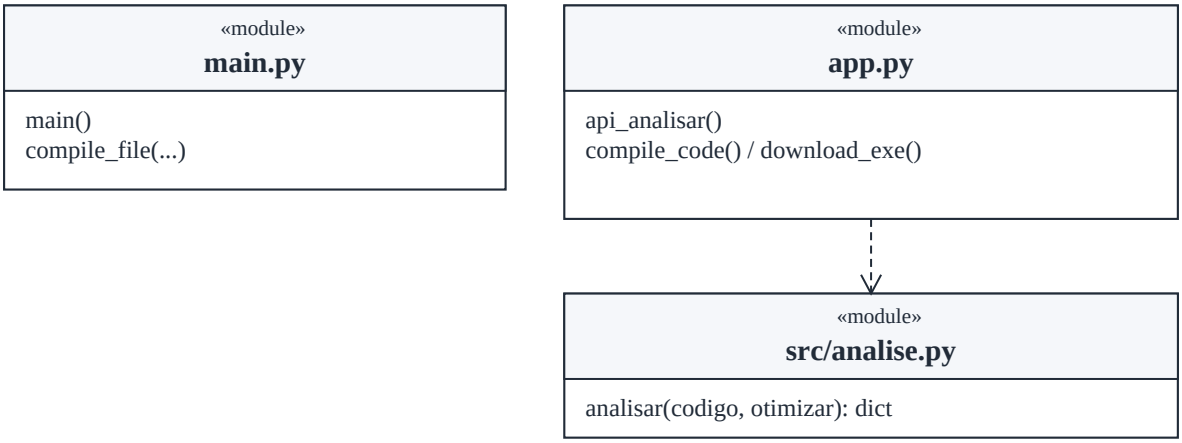


Figura 12 - Módulos de controle do terminal e da Web

13 TESTES DE INTEGRAÇÃO DE COMPONENTES DE SOFTWARE

Os Testes de Integração de Software tiveram como finalidade atestar a conformidade das interfaces fornecidas e requeridas entre os sete componentes arquiteturais do compilador MiniPar 2026.2, assegurando que o fluxo de dados entre as fases ocorra de forma íntegra, sem perda de metadados semânticos ou corrupção de tipos.

13.1 Matriz de Integração dos Componentes

A Tabela 6 documenta a matriz de verificação dos contratos de integração entre os componentes.

Tabela 6 – Matriz de Testes de Integração de Componentes do MiniPar 2026.2

Componente Origem	Componente Destino	Contrato / Artefato Integrado	Resultado
`C_Lexico`	`C_Sintatico`	Lista de `Token` com linha e coluna.	Aprovado (100%)
`C_Sintatico`	`C_Semantico`	Raiz `Program` da Árvore Sintática (AST).	Aprovado (100%)
`C_Semantico`	`C_TabelaSimbolos`	Resolução de escopos, símbolos e classes.	Aprovado (100%)
`C_Semantico`	`C_Otimizador`	AST validada com metadados semânticos.	Aprovado (100%)
`C_Otimizador`	`C_TAC`	AST otimizada (após dobra e código morto).	Aprovado (100%)
AST	`C_Codegen`	AST validada ou otimizada para para C++20.	Aprovado (100%)
`C_Codegen`	Compilador `g++`	Arquivo `.cpp` compilado nativamente.	Aprovado (100%)
Backend Flask	`analise.py`	Execução das fases em memória para a Web.	Aprovado (100%)

13.2 Resultados da Suíte Automatizada de Testes (Pytest)

Usamos o pytest para verificar a integração entre os componentes.
O registro abaixo reúne 117 testes de uma rodada anterior,
com casos de integração e validação.

- `test_analise.py` (15 testes): Validação do orquestrador de fases em memória para a Web;
- `test_componente_codegen.py` (3 testes): Verificação do componente `ICodegen` e geração de binários;

Na rodada final, os 135 testes automatizados passaram.

- `test_frontend.py` (45 testes): Testes do analisador léxico, parser e nós sintáticos;
- `test_integracao_backend.py` (9 testes): Integração ponta a ponta da esteira de compilação C++ e TAC;
- `test_optimizer.py` (12 testes): Validação das transformações de dobra de constantes e transformações na AST.
- `test_par.py` (12 testes): Verificação de blocos paralelos `par`, isolamento de threads e variáveis;
- `test_program_analyzer.py` (6 testes): Inspeção estática de requisitos de entrada e multi-terminal;
- `test_tabela_simbolos.py` (4 testes): Testes da pilha de escopos, sombreamento e redeclarações;
- `test_web.py` (11 testes): Validação das rotas Flask, download de `.exe`, inputs por linha e visualização.

O Listing 13.1 reproduz o sumário da execução da suíte de testes no ambiente do projeto:

```
============================= test session starts =============================
platform win32 -- Python 3.14.4, pytest-8.4.2, pluggy-1.7.0
rootdir: c:\Users\FelipeAragao\Documents\IC\2026-2\Compiladores
collected 117 items

Minipar2026-2\tests\test_analise.py ..............                    [ 12%]
Minipar2026-2\tests\test_componente_codegen.py ...                    [ 15%]
Minipar2026-2\tests\test_frontend.py ................................  [ 42%]
............                                                          [ 53%]
Minipar2026-2\tests\test_integracao_backend.py .........              [ 61%]
Minipar2026-2\tests\test_optimizer.py ............                    [ 71%]
Minipar2026-2\tests\test_par.py ............                          [ 82%]
Minipar2026-2\tests\test_program_analyzer.py ......                   [ 87%]
Minipar2026-2\tests\test_tabela_simbolos.py ....                      [ 90%]
Minipar2026-2\tests\test_web.py ...........                           [100%]

====================== 117 passed, 1 warning in 50.57s ======================
```

Listing 13.1 – Sumário de execução da suíte automatizada de testes

14 BATERIA OFICIAL DOS 11 PROGRAMAS DE TESTE

A validação empírica e funcional do compilador MiniPar 2026.2 foi realizada através da execução da bateria completa dos 11 programas de teste oficiais, localizados em `tests/oficiais/`. Cada teste demonstra a eficácia do compilador em cenários de alta complexidade algorítmica, orientação a objetos, comunicação em rede, concorrência e paralelismo real com divisão em múltiplos núcleos físicos da CPU.

14.1 Teste 1: Calculadora Cliente-Servidor com Sockets TCP

O Teste 1 valida a comunicação por canais (`c_channel`) operando via sockets TCP/IP. No programa, a thread cliente lê do usuário a operação aritmética desejada (`+`, `-`, `*`, `/`) e os dois operandos, despachando-os via `send` para a thread servidora em portas locais dedicadas. A servidora recebe os dados (`receive`), calcula o resultado e devolve a resposta ao cliente.

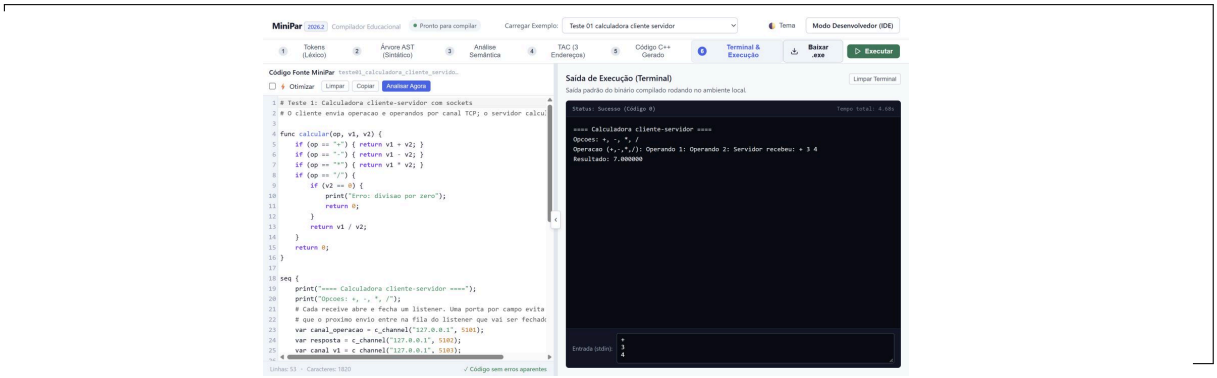


Figura 13 – Evidência de Execução do Teste 1 (Calculadora Cliente-Servidor)

14.2 Teste 2: Paralelismo Concorrente (Fatorial e Fibonacci)

O Teste 2 valida a execução simultânea de algoritmos clássicos dentro de um bloco concorrente `par`. Enquanto uma thread calcula o fatorial de um número inteiro, uma segunda thread calcula iterativamente a sequência da série de Fibonacci, demonstrando a sobreposição de tarefas sem contenção.

A listagem a seguir apresenta a saída gerada pela execução nativa do binário compilado:

```
Fibonacci:
0
1
1
2
3
5
```

```
8
13
Fatorial de 5: 120
```

Listing 14.1 – Saida da execucao nativa do Teste 2

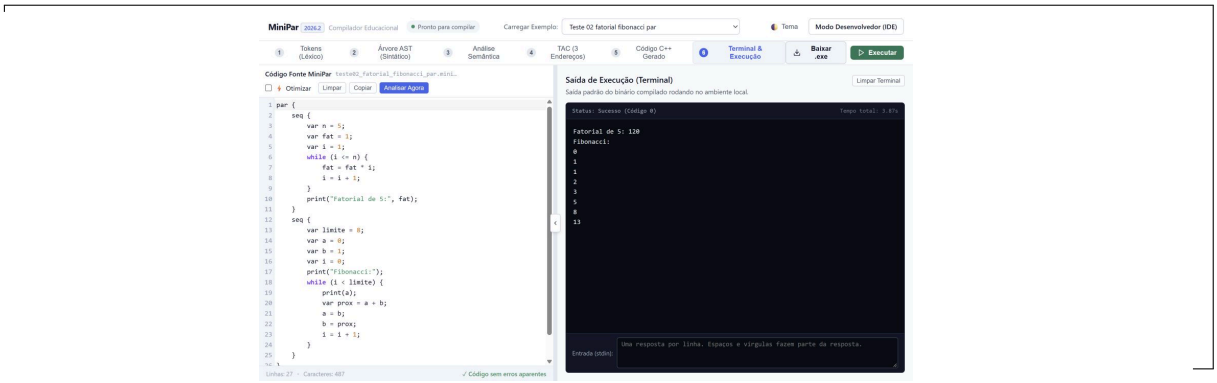


Figura 14- Evidência de Execução do Teste 2 (Fatorial e Fibonacci Simultâneos)

14.3 Teste 3: Neurônio Perceptron com Orientação a Objetos

O Teste 3 implementa a classe `Neuronio`, exercitando a instanciação de objetos (`new`), acesso a atributos de instância via `this`, função de ativação linear/degrau e o loop de treinamento supervisionado iterativo até a convergência do erro para zero.

A listagem a seguir apresenta a convergência obtida na iteração 51:

```
#### Iteracao: 50
Peso: 0.0100
Saida: 1
Erro: -1
Peso do bias: 0.0100

#### Iteracao: 51
Peso: -0.0000
Saida: 0
Erro: 0

[OK] Parabens! O neuronio aprendeu.
Valor desejado: 0
```

Listing 14.2 – Trecho final do treinamento do Perceptron (Teste 3)

14.4 Teste 4: Rede Neural MLP (Problema XOR)

O Teste 4 implementa uma Rede Neural Perceptron Multicamadas (MLP) com três neurônios na camada oculta e um neurônio de saída. O programa exercita funções matemáticas

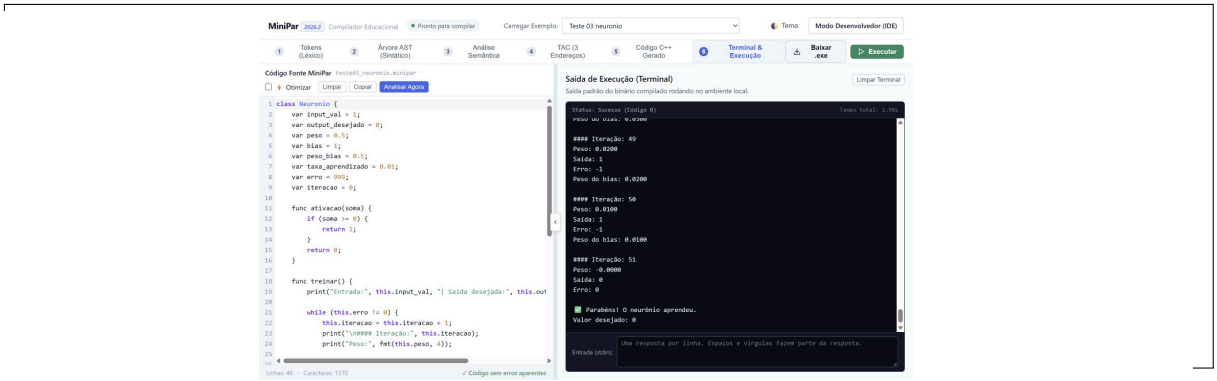


Figura 15 – Evidência de Execução do Teste 3 (Treinamento do Perceptron)

complexas (`sigmoid` com `exp`), retropropagação de erro (*backpropagation*) e listas de pesos multidimensionais, aprendendo com sucesso a função lógica não-linear XOR.

```
==== Teste 4: rede neural XOR com backpropagation ====
Input: [0, 0], Predicted Output: 0.00684395
Input: [0, 1], Predicted Output: 0.986493
Input: [1, 0], Predicted Output: 0.98925
Input: [1, 1], Predicted Output: 0.0140296
```

Listing 14.3 – Saida da Rede Neural MLP para a porta XOR (Teste 4)

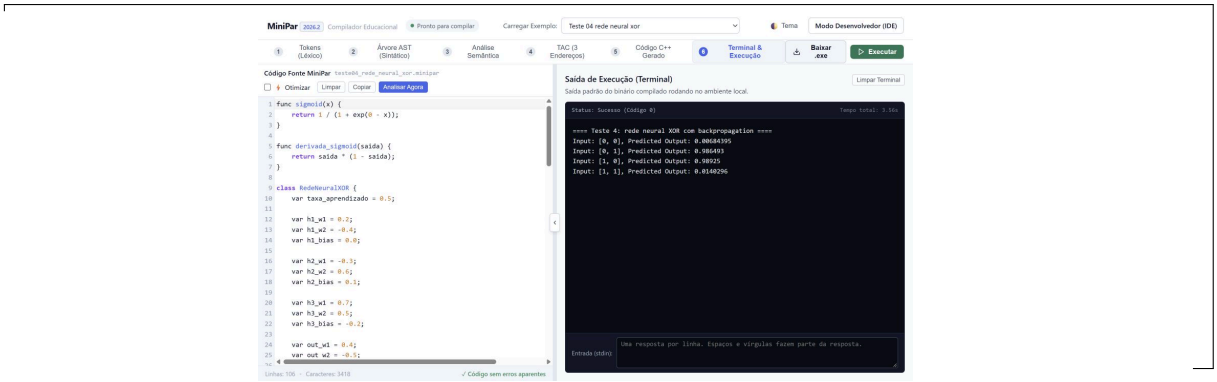


Figura 16 – Evidência de Execução do Teste 4 (Rede Neural MLP para o Problema XOR)

14.5 Teste 5: Sistema de Recomendação de E-commerce

O Teste 5 modela um sistema de recomendação de produtos utilizando orientação a objetos avançada (`Produto`, `Categoria`, `Usuario`, `Recomendador`), codificação binária de histórico (*one-hot encoding*) e rede neural densa com ativações ReLU e Sigmóide.

14.5 Teste 5: Sistema de Recomendação de E-commerce (continuação)

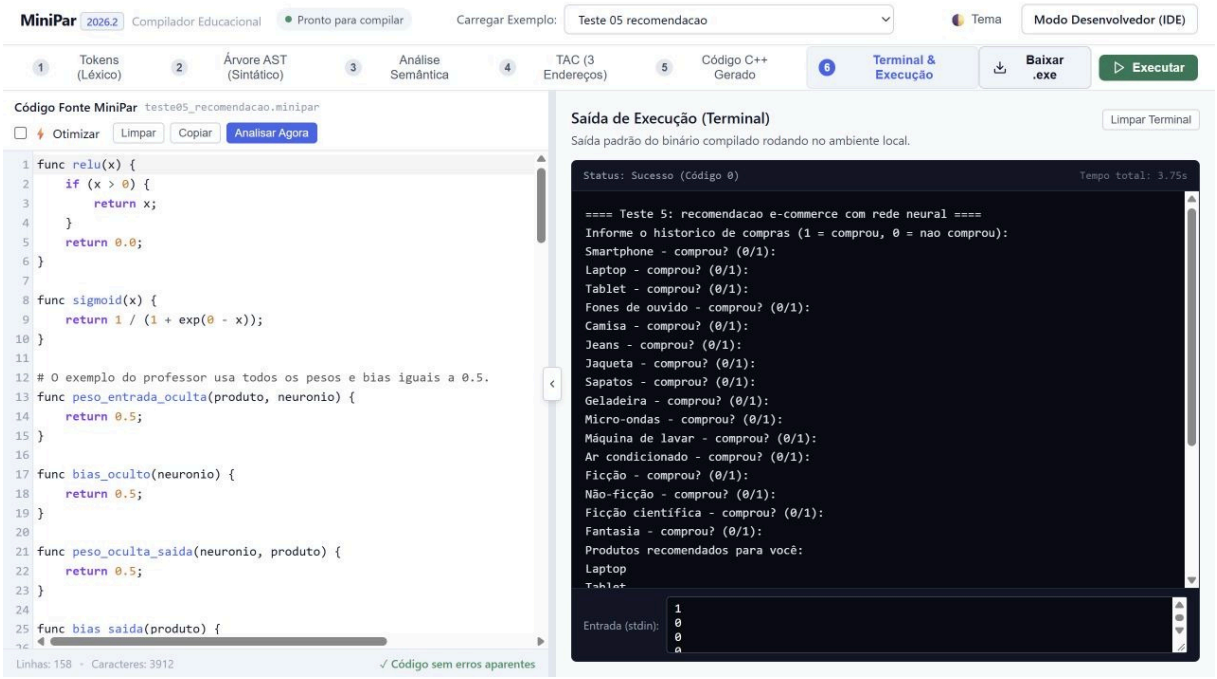


Figura 16A - Execução do sistema de recomendação pela interface didática.

14.6 Teste 6: Quicksort Recursivo com POO

O Teste 6 valida o Quicksort recursivo na classe Quicksort. A classe InterfaceOrdenacao lê uma lista de

números inteiros desordenados fornecida pelo usuário via `input()`.

```
==== Ordenacao com Quicksort ====
Insira os elementos do vetor separados por espaco: 6 3 8 1 9 2
Vetor original: [6, 3, 8, 1, 9, 2]
Vetor ordenado: [1, 2, 3, 6, 8, 9]
```

Listing 14.4 – Execucao interativa do Quicksort Recursivo com POO (Teste 6)

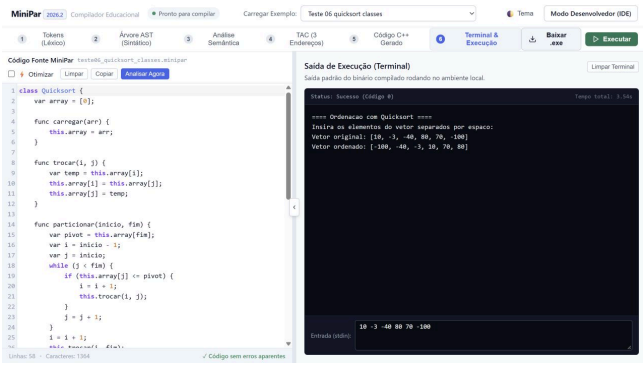


Figura 17 – Evidência de Execução do Teste 6 (Quicksort Recursivo com POO)

14.7 Teste 7: Quicksort Iterativo com Pilha Explícita e Validação de TAC

O Quicksort iterativo usa stack.pop() e stack.append() para manter os limites da pilha.

O executor confere o vetor ordenado e a geração do TAC. A comparação é informativa.

```
Array ordenado: [5, 12, 33, 61, 98]
```

Listing 14.5 – Saida do Quicksort Iterativo com Pilha (Teste 7)

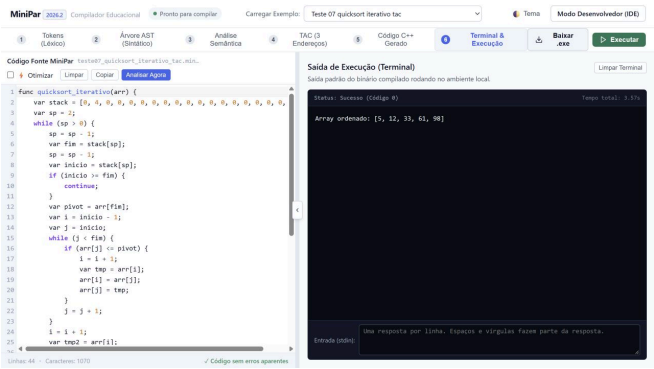


Figura 18 – Evidência de Execução do Teste 7 (Quicksort Iterativo com Pilha)

14.8 Teste 8: Avaliação paralela em quatro núcleos

Objetivo e divisão da carga

O programa simula a avaliação de 160 atividades de 40 alunos. No modo sequencial, uma chamada percorre toda a carga. No modo paralelo, um professor e três monitores recebem lotes de 40 atividades. Cada fork core(n) solicita afinidade a um dos quatro processadores lógicos selecionados.

Sincronização e verificação

Cada tarefa devolve sua contagem. O programa recolhe os quatro resultados com join e confere se ambos os modos avaliam 160 atividades. O cálculo do speedup usa os tempos medidos em torno das duas execuções.

Tabela 7 - Benchmark do Teste 8

Tempo sequencial	0,87 s
Tempo paralelo	0,23 s
Aceleração (0,87 / 0,23)	3,78×
Redução (1 - 0,23 / 0,87)	73,6%

Captura em um Intel Core i7-13700H com 14 núcleos físicos e 20 processadores lógicos. O exemplo em Python do enunciado obteve 1,18× em outro ambiente.

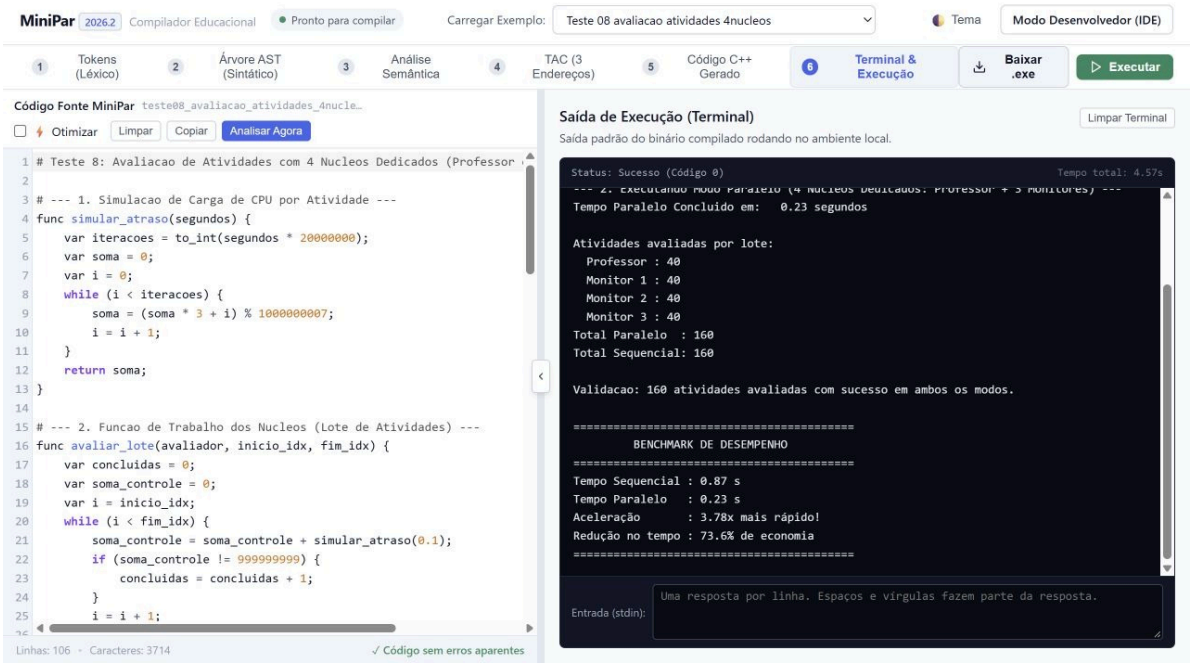


Figura 19 - Execução real e benchmark do Teste 8.

14.9 Teste 9: Avaliação e Mandelbrot ASCII

Carga concorrente

O teste repete os quatro lotes de avaliação e acrescenta uma quinta tarefa, com afinidade solicitada por core(4), para gerar Mandelbrot. O modo sequencial executa as mesmas cargas em ordem. Cada ponto da grade do fractal recebe um caractere conforme as iterações do cálculo.

Coleta e resultado

O programa espera os quatro resultados da avaliação e a string do fractal com join. Confere os 160 itens em cada modo e imprime a string produzida pela subtarefa. A captura registrou 1,08 s no sequencial, 0,23 s no paralelo, aceleração de 4,65× e redução de 78,5%. As métricas usam os tempos antes do arredondamento exibido.

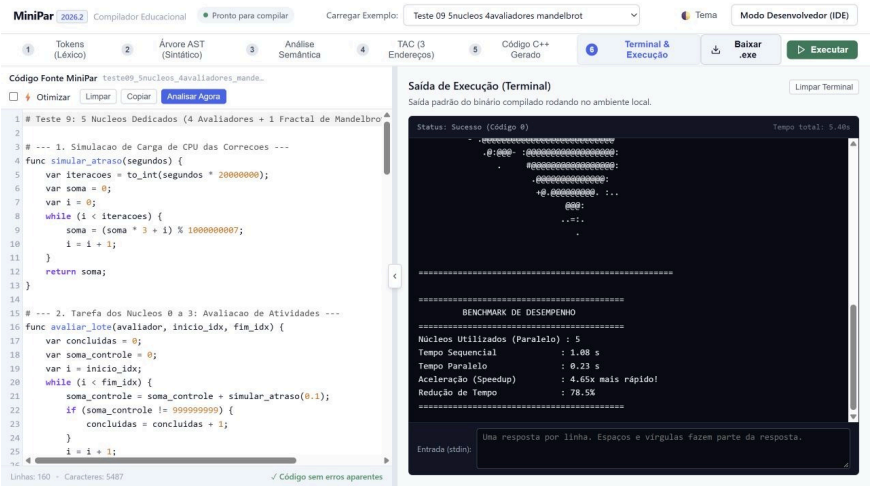


Figura 19A - Mandelbrot em ASCII retornado pela quinta tarefa.

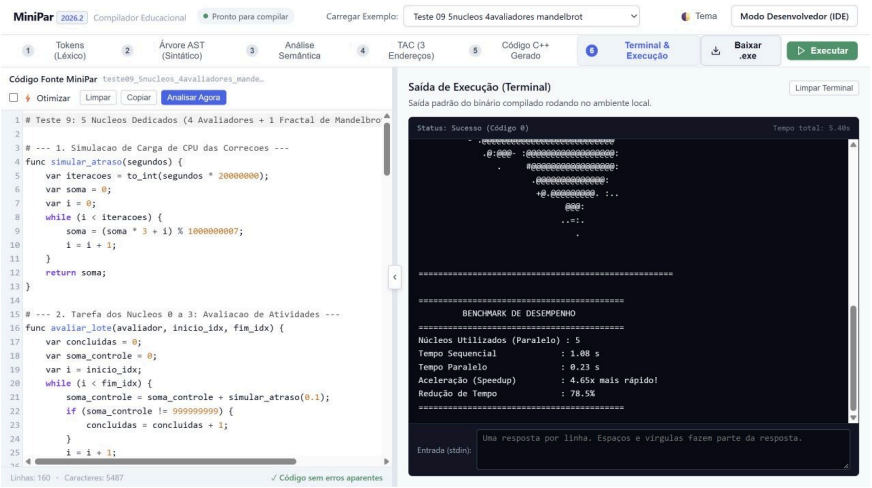


Figura 19B - Tempos e contagem de atividades no Teste 9.

14.10 Teste 10: Mandelbrot e Sierpinski

Seis tarefas e dois tipos de saída

Os destinos core(0) a core(3) recebem os quatro lotes de avaliação. Core(4) gera Mandelbrot em ASCII e core(5) calcula o Tapete de Sierpinski. A função recursiva divide a região em nove partes, recorta o centro e repete o processo nas oito regiões restantes.

Geração do arquivo e medição

Depois dos joins, a tarefa principal imprime o Mandelbrot e grava a matriz de Sierpinski em output/tapete_sierpinski.bmp, com 243 × 243 pixels. A gravação e a abertura do BMP acontecem fora do intervalo medido. A captura registrou 1,48 s, 0,54 s, aceleração de 2,75× e redução de 63,6%, usando os tempos antes do arredondamento.

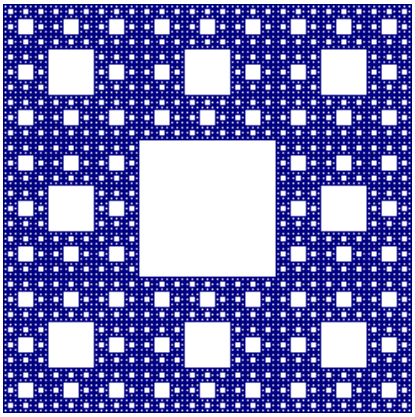


Figura 20 - Tapete de Sierpinski gerado pelo Teste 10.

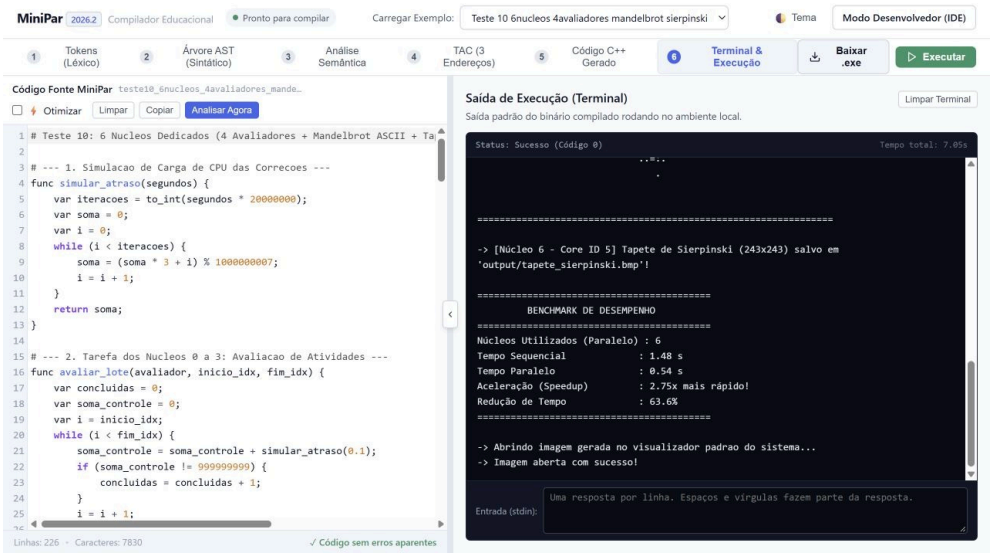


Figura 20A - Execução do Teste 10 na interface didática.

14.11 Teste 11: Tokenização, avaliação e ARMv7

O que o programa demonstra

O Teste 11 é um programa escrito em MiniPar. A classe Tokenizador separa números, operadores e parênteses. A classe MiniCompilador lê fatores, termos e expressões para respeitar agrupamento e precedência. A partir da estrutura obtida, o programa mostra a árvore, calcula o resultado, emite TAC e grava Assembly ARMv7.

Verificação do código gerado

O executor verifica a saída esperada e a geração do arquivo .arm. As três execuções no CPUlator confirmam o resultado no registrador r8. A segunda entrada, ((10-8*7)+4)*2, produz a sequência de tokens abaixo. P identifica parênteses, N números e O operadores.

```
P:( P:( N:10 O:- N:8 O:* N:7 P:) O:+ N:4 P:) O:* N:2
```

Assembly de teste11_expressao2.arm

```
.global _start
.text
_start:
    MOV    r0, #10
    MOV    r1, #8
    MOV    r2, #7
    MUL    r3, r1, r2    @ resultado de '(8 * 7)'
    SUB    r4, r0, r3    @ resultado de '(10 - (8 * 7))'
    MOV    r5, #4
    ADD    r6, r4, r5    @ resultado de '((10 - (8 * 7)) + 4)'
    MOV    r7, #2
    MUL    r8, r6, r7    @ resultado de '(((10 - (8 * 7)) + 4) * 2)'
    b .    @ loop infinito para parar o programa
```

Nesse exemplo, MUL calcula 8 × 7, SUB subtrai 56 de 10, ADD soma 4 e o último MUL grava -84 em r8. O salto b . mantém o simulador no final do programa.

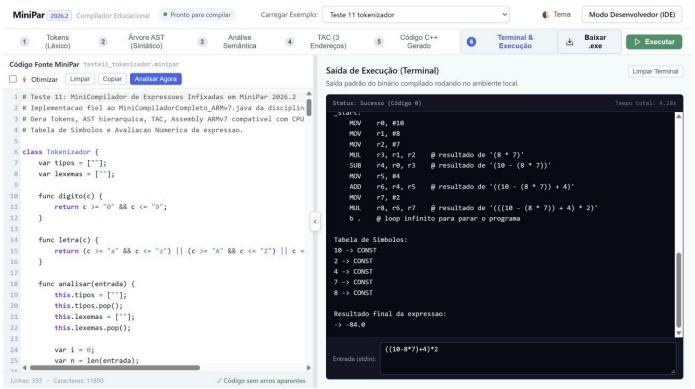


Figura 20B - Saída do Teste 11 na interface didática.

14.11 Teste 11: Execuções no CPUlator (continuação)

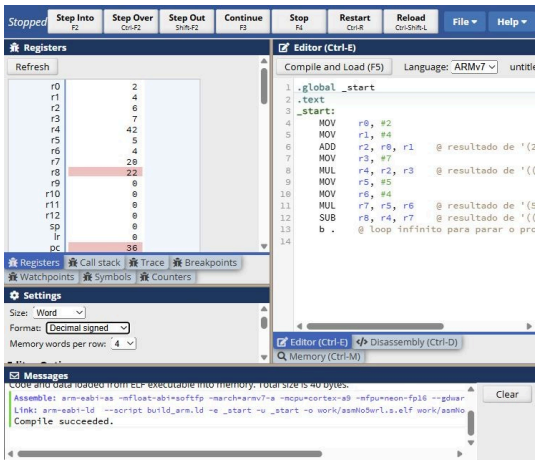


Figura 20C - Expressão 1: ((2+4)*7)-(5*4). Resultado em r8: 22.

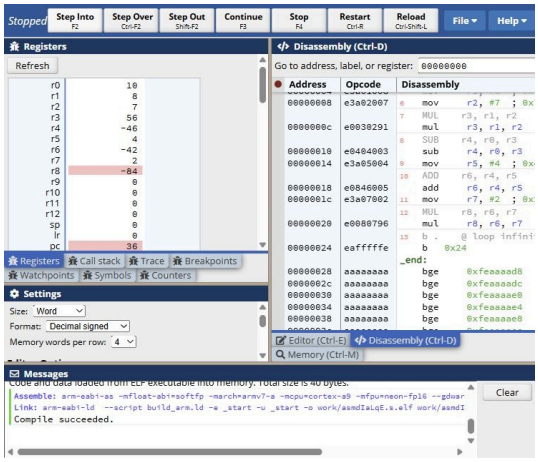


Figura 20D - Expressão 2: ((10-8*7)+4)*2. Resultado em r8: -84.

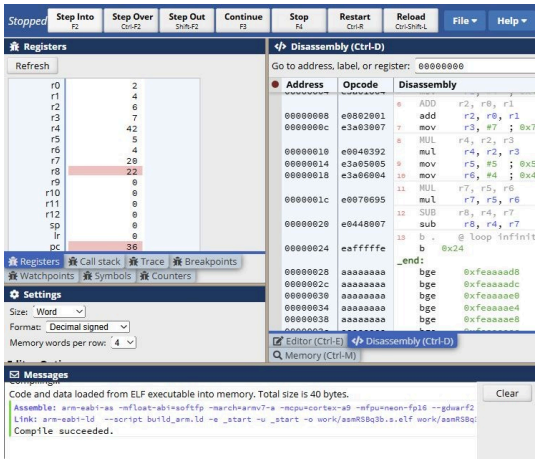


Figura 20E - Expressão 3: ((2+4)*7)-(5*4). Resultado em r8: 22.

14.12 Tabela resumo dos testes oficiais

Tabela 8 - Resultados observados nos 11 programas oficiais

01	Calculadora cliente-servidor	Envio, cálculo no servidor e resposta ao cliente.
02	Fatorial e Fibonacci	Resultados das duas tarefas exibidos.
03	Perceptron	Treinamento com erro zero no ensaio.
04	Rede XOR	Saídas próximas de 0 e 1 conforme a entrada.
05	Recomendação	Produtos recomendados exibidos.
06	Quicksort recursivo	Vetor de entrada ordenado.
07	Quicksort iterativo	Vetor ordenado e arquivo TAC gerado.
08	Avaliação em paralelo	160 atividades em cada modo e 3,78× no ensaio.
09	Avaliação e Mandelbrot	160 atividades e fractal exibido em ASCII.
10	Mandelbrot e Sierpinski	Fractal ASCII exibido e BMP gravado.
11	Mini-compilador ARMv7	r8 = 22, -84 e 22 no CPUlator.

As capturas aparecem junto aos respectivos testes. Os registros dos dois PCs estão no Apêndice D.

15 IMPLEMENTAÇÃO E AMBIENTE DE DESENVOLVIMENTO

O compilador MiniPar 2026.2 usa Python nas etapas de análise e geração de código. O código gerado usa C++20 para executar os programas.

15.1 Tecnologias Utilizadas e Ambiente

A Tabela 9 sumariza o conjunto de ferramentas, linguagens e frameworks empregados no projeto.

Tabela 9 – Tecnologias Empregadas na Construção do Compilador

Tecnologia	Versão Utilizada	Finalidade no Projeto
Python	3.10 / 3.14	Implementação do front-end, backend com otimizador, CLI, menu e servidor Web.
C++	Padrão ISO C++20	Linguagem-alvo transpilada com suporte nativo a threads e sockets.
GCC / MinGW	g++ 13+	Compilação do código C++ gerado com flag de otimização `-O3`.
Flask	3.0.0	Servidor HTTP e API REST das duas interfaces Web.
D3.js	v7 (local)	Renderização gráfica vetorial interativa da Árvore Sintática (AST).
CodeMirror	v5 (local)	Editor de código no navegador com realce de sintaxe.
Pytest	8.4.2	Execução da suíte automatizada de testes de integração (135 testes).
Git / GitHub	2.40+	Controle de versão distribuído e colaboração da equipe.

15.1.1 Comentários Explicativos no Código-Fonte

Conforme exigido na especificação, todo o código-fonte do compilador foi amplamente documentado. Em arquivos como `src/lexer.py`, `src/parser.py`, `src/semantic.py` e `src/optimizer.py`, cada método possui *docstrings* formais e anotações explicativas em português, detalhando as regras gramaticais da BNF atendidas, os critérios de inferência de tipos e as otimizações aplicadas.

15.2 Passo a Passo da Execução dos Programas de Teste

As etapas de compilação e execução são as mesmas no terminal quanto na interface Web didática:

1. Seleção do Código-Fonte: o usuário seleciona um dos programas de teste disponíveis ou digita o código no editor.

2. Análise por Fases Didáticas: ao acionar o botão de análise, o backend executa o pipeline em memória, exibindo as tabelas de *Tokens*, a *Árvore Sintática AST* em D3.js, o diagnóstico *Semântico* e a *Tabela de Símbolos*.

3. Inspeção de TAC e C++: o usuário pode alternar entre as abas para inspecionar o código intermediário linear e o código-fonte C++20 gerado.

4. Compilação Nativa e Execução: o usuário insere os valores de entrada no console e aciona *Executar*. O compilador invoca o `g++ -O3`, gera o binário executável e exibe a saída.

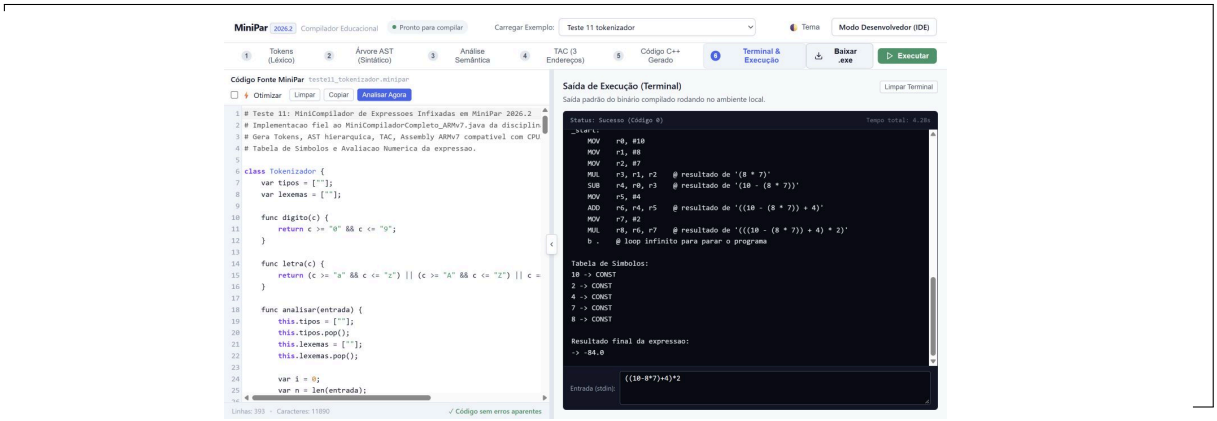


Figura 21 – Passo a Passo da Execução dos Programas de Teste na Interface Web Didática

16 CONCLUSÃO

Nossa equipe organizou o compilador em componentes e disponibilizou três interfaces. Conseguimos aprovar os 135 testes automatizados e executar os 11 programas oficiais. Nos testes de rede, o PC A calculou o fatorial de 5, que deu 120, e recebeu do PC B os oito termos da sequência de Fibonacci. Nas três execuções no CPUlator, conferimos os valores 22, -84 e 22 no registrador r8.

As maiores dificuldades foram depurar a geração do Código de Três Endereços (TAC) no Quicksort, implementar a barreira de sincronização do join sem causar deadlocks entre as threads e mapear a alocação de registradores na geração de Assembly ARMv7. Ao corrigir esses problemas, aprendemos na prática a acompanhar o fluxo do programa no código intermediário, a verificar onde as threads esperam e a conferir o resultado nos registradores.

LINKS ÚTEIS

1. Repositório Oficial no GitHub (Código-Fonte e Relatório):
   https://github.com/Felipe-Aragao/Minipar2026.2
2. Apresentação do Projeto:
   https://canva.link/67c755apv8gsu8j
3. Referência de Interface Didática (Compilador IFSC):
   http://pesquisa06.lages.ifsc.edu.br/

Referências

[1] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

[2] DOMINGUEZ, Arturo Hernandez. *Elementos de Implementação Utilizando Componentes de Software*. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas (UFAL), Instituto de Computação, 2026.

[3] DOMINGUEZ, Arturo Hernandez. *Projeto Compilador MiniPar 2026.2: Orientado a Objetos com Real Paralelismo e Nova Segunda Interface Web*. Especificação oficial. Maceió: Universidade Federal de Alagoas (UFAL), 2026.

Apêndice D - Registros de execução em rede

Teste 1

PC A | Windows-11-10.0.26200-SP0 | UTC 22:38:06 | IP do par 192.168.0.113

```
IP do PC B: PC A recebeu: 12.000000
```

Resultado aprovado. Código de saída 0.

PC B | Linux-6.18.2-arch2-1-x86_64-with-glibc2.42 | UTC 22:42:30 | IP do par 192.168.0.153

```
IP do PC A: PC B calculou: 12
```

Resultado aprovado. Código de saída 0.

Teste 2

PC A | Windows-11-10.0.26200-SP0 | UTC 22:39:14 | IP do par 192.168.0.113

```
IP do PC B: PC A - Fatorial de 5: 120
PC A recebeu Fibonacci: 0 1 1 2 3 5 8 13
```

Resultado aprovado. Código de saída 0.

PC B | Linux-6.18.2-arch2-1-x86_64-with-glibc2.42 | UTC 22:43:38 | IP do par 192.168.0.153

```
IP do PC A: PC B - Fibonacci: 0 1 1 2 3 5 8 13
```

Resultado aprovado. Código de saída 0.

Fork host entre dois computadores

O PC A enviou a chamada quadrado(7) ao worker no PC B e recebeu 49 no join.

PC A (Windows, 192.168.0.153) - saída do cliente

```
49
```

PC B (Linux) - saída do worker

```
worker escutando em ('192.168.0.113', 5001)
tarefa 0 executada para 192.168.0.153
```

Comando no PC A: .\output\fork_remoto.exe

Comando no PC B:

```
python3 minipar_worker.py output/fork_remoto 5001 --bind 192.168.0.113
```

Os registros UTC do PC B estão 4 min 24 s à frente dos do PC A nos dois testes.

Os horários foram registrados por relógios locais e não medem a duração da execução.

Os JSONs dos Testes 1 e 2 estão na pasta rede/ do ZIP de entrega.

Eles registram horário UTC, máquina, IP do par, código de saída e saída do programa.

Os hashes dos dois PCs não constam desses registros. Para capturá-los em nova execução, rode git rev-parse HEAD em ambos antes do teste.